

The Siemens logo is displayed in a bold, teal, sans-serif font.

SIEMENS

Ingenuity for life

A man in a light blue shirt is seen from the side, holding a tablet. Overlaid on the image are various digital graphics: a '24/7' icon with a circular arrow, a 'NEWS' section with a person icon, a 'Home' button, and a network diagram with three nodes. The background is a blurred industrial setting with a clock on the wall.

Guideline on Library Handling in TIA Portal

TIA Portal

<https://support.industry.siemens.com/cs/ww/en/view/109747503>

Siemens
Industry
Online
Support



Legal information

Use of application examples

Application examples illustrate the solution of automation tasks through an interaction of several components in the form of text, graphics and/or software modules. The application examples are a free service by Siemens AG and/or a subsidiary of Siemens AG ("Siemens"). They are non-binding and make no claim to completeness or functionality regarding configuration and equipment. The application examples merely offer help with typical tasks; they do not constitute customer-specific solutions. You yourself are responsible for the proper and safe operation of the products in accordance with applicable regulations and must also check the function of the respective application example and customize it for your system.

Siemens grants you the non-exclusive, non-sublicensable and non-transferable right to have the application examples used by technically trained personnel. Any change to the application examples is your responsibility. Sharing the application examples with third parties or copying the application examples or excerpts thereof is permitted only in combination with your own products. The application examples are not required to undergo the customary tests and quality inspections of a chargeable product; they may have functional and performance defects as well as errors. It is your responsibility to use them in such a manner that any malfunctions that may occur do not result in property damage or injury to persons.

Disclaimer of liability

Siemens shall not assume any liability, for any legal reason whatsoever, including, without limitation, liability for the usability, availability, completeness and freedom from defects of the application examples as well as for related information, configuration and performance data and any damage caused thereby. This shall not apply in cases of mandatory liability, for example under the German Product Liability Act, or in cases of intent, gross negligence, or culpable loss of life, bodily injury or damage to health, non-compliance with a guarantee, fraudulent non-disclosure of a defect, or culpable breach of material contractual obligations. Claims for damages arising from a breach of material contractual obligations shall however be limited to the foreseeable damage typical of the type of agreement, unless liability arises from intent or gross negligence or is based on loss of life, bodily injury or damage to health. The foregoing provisions do not imply any change in the burden of proof to your detriment. You shall indemnify Siemens against existing or future claims of third parties in this connection except where Siemens is mandatorily liable.

By using the application examples you acknowledge that Siemens cannot be held liable for any damage beyond the liability provisions described.

Other information

Siemens reserves the right to make changes to the application examples at any time without notice. In case of discrepancies between the suggestions in the application examples and other Siemens publications such as catalogs, the content of the other documentation shall have precedence.

The Siemens terms of use (<https://support.industry.siemens.com>) shall also apply.

Security information

Siemens provides products and solutions with Industrial Security functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial security concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the Internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place. For additional information on industrial security measures that may be implemented, please visit <https://www.siemens.com/industrialsecurity>.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Security RSS Feed at: <https://www.siemens.com/industrialsecurity>.

Table of contents

Legal information	2
1 Introduction	5
1.1 The type-instance concept	5
1.1.1 Purpose of the guideline.....	5
1.1.2 Advantages of the type-instance concept	5
1.1.3 Using the type-instance concept	5
1.1.4 Applicability of this guide	6
1.2 Libraries and library elements in TIA Portal	7
1.3 Working with types and master copies.....	8
1.3.1 Storing a block as master copy in project library	8
1.3.2 Adding a block to project as master copy	9
1.3.3 Storing a block as a type in the project library	10
1.3.4 Using typified blocks in a project.....	11
1.3.5 Copying blocks into global library as master copy / type	11
1.3.6 Editing a type.....	12
1.3.7 Handling with default version	14
1.3.8 Additional fundamentals	15
1.4 Central storage location for global libraries.....	15
2 Creating a corporate library	16
2.1 Programming guideline	16
2.2 Workflow.....	16
2.3 "Creation / maintenance" process.....	17
2.3.1 "Integrator" role	17
2.3.2 "Developer" role	17
2.3.3 "Integrator project" (area of responsibility of integrator).....	18
2.3.4 "Development library" (area of responsibility of integrator)	18
2.3.5 "Developer project(s)" (area of responsibility of developer)	18
2.3.6 "Transfer library or libraries" (area of responsibility of developer)	18
2.3.7 "Creation / maintenance" development cycle.....	19
2.3.8 Alternative: Library development in a Multiuser project	20
2.4 "Publication" process.....	21
2.4.1 "Integrator" role	21
2.4.2 Corporate library (area of responsibility of integrator).....	21
2.5 "Use" process.....	23
2.5.1 "User" role	23
2.5.2 Plant projects (area of responsibility of user)	23
3 Applications in TIA Portal for the "integrator"	24
3.1 Working with the "integrator project"	24
3.1.1 Adding new library elements from "transfer libraries" to the "integration project"	24
3.1.2 Updating existing library elements from the "transfer libraries" in the "integrator project"	28
3.1.3 Tips for structuring the container FB	32
3.2 Working with the global "development library"	33
3.2.1 Creating the global "development library"	33
3.2.2 Adding and updating blocks from "integrator project" into the global "development library"	34
3.2.3 Deleting older versions of types in the global "development library".	35
3.3 Working with the global "corporate library"	37
3.3.1 Creating and approving the global "corporate library"	37
3.3.2 Creating write-protected libraries	38
3.3.3 Discontinuation of types	40

4	Applications in TIA Portal for "developers"	42
4.1	Working with the "developer project"	42
4.1.1	Updating the "developer project" with the global "development library"	43
4.1.2	Creating new typified elements	47
4.1.3	Editing typified elements	50
4.2	Working with the global "transfer library"	52
4.2.1	Creating a global "transfer library"	52
4.2.2	Copying typified elements into the global "transfer library"	52
5	Applications in TIA Portal for "users"	54
5.1	Working with the "plant project"	54
5.1.1	Opening the archived global "corporate library"	54
5.1.2	Adding typified library elements to "plant project"	55
5.1.3	Updating selected types in the "plant project"	56
5.1.4	Updating all types in the "plant project"	58
5.1.5	Replacing discontinued types	61
6	Creating customized libraries	66
7	Troubleshooting	68
7.1	Inconsistent library status	68
7.1.1	Inconsistency in the default versions	69
7.1.2	Inconsistency with same versions ("duplicate version")	72
7.1.3	"Non Default Version" inconsistency	74
7.1.4	"Multiple Versions Instantiation" inconsistency	75
7.2	Resolving multiple versions	76
7.3	Undesired assignment of a new version	80
7.4	Undesired disconnection of an instance from its type	81
7.5	Undesired deletion of a type instance	81
7.6	Deleted types in global libraries are not deleted from project	81
8	Useful information / fundamentals	82
8.1	Terms	82
8.2	Overview of all library elements	83
8.3	Differences in types between PLC and HMI	83
8.4	Displaying type information in the project tree	84
8.5	Library view and Library management	86
8.5.1	Overview	86
8.5.2	Filter dialog in the Library management	87
8.5.3	Showing and hiding columns in the "Libraries" task card	88
8.5.4	Filtering library elements in the project library	89
8.5.5	Harmonize project and libraries	91
8.6	Version numbers	93
8.7	Know-how protection	93
8.8	Write protection	94
8.9	User-defined documentation (User help)	94
8.9.1	Creating user-defined documentation	94
8.9.2	Accessing user-defined documentation	96
8.10	Enabling simulation support for library blocks	97
9	Appendix	99
9.1	Service and support	99
9.2	Industry Mall	100
9.3	Links and literature	100
9.4	Change documentation	101

1 Introduction

1.1 The type-instance concept

1.1.1 Purpose of the guideline

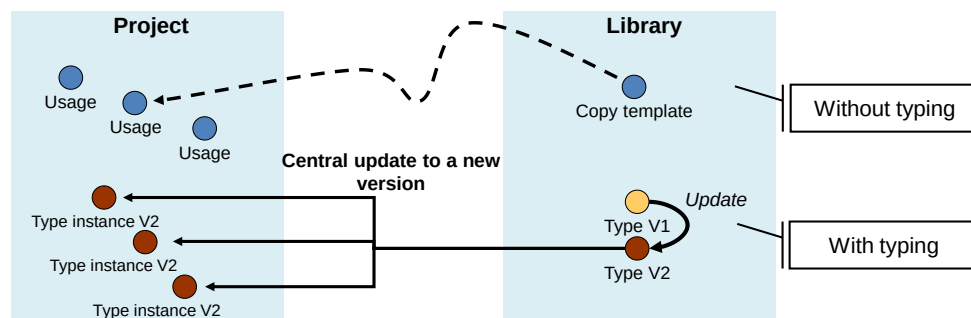
This guide describes how to create a corporate library with the use of typified library elements in TIA Portal.

You have the option of creating types or master copies from TIA Portal elements (e.g. function blocks) in a library (see [Figure 1-1: Typification with libraries](#)). When they are used, types become "instantiated" in the project. All type instances have a fixed connection to their type in the library. In this way, you can version library elements and make changes centrally; changes will be refreshed throughout the entire project and are supported by the system.

Master copies are independent copies of TIA Portal elements. They have no connection to their instances of use in the project, nor do they have system-supported versioning or central editability.

These differences should always inform your decision whether to create types or master copies in libraries.

Figure 1-1: Typification with libraries



1.1.2 Advantages of the type-instance concept

This guide recommends the use of types because of the following advantages:

- Central refresh function of all instances in the project
- Central display of the instances where types are used
- System-supported version management of library elements
- System-supported change tracking of library elements
- Consistent library elements for multiple users
- Typified elements (type instances) are specially marked in a project

1.1.3 Using the type-instance concept

A defined workflow would be necessary in order to exhaustively cover the advantages of the type-instance concept. Chapter [2](#), entitled [Creating a corporate library](#) describes the optimum workflow.

1.1.4 Applicability of this guide

TIA Portal

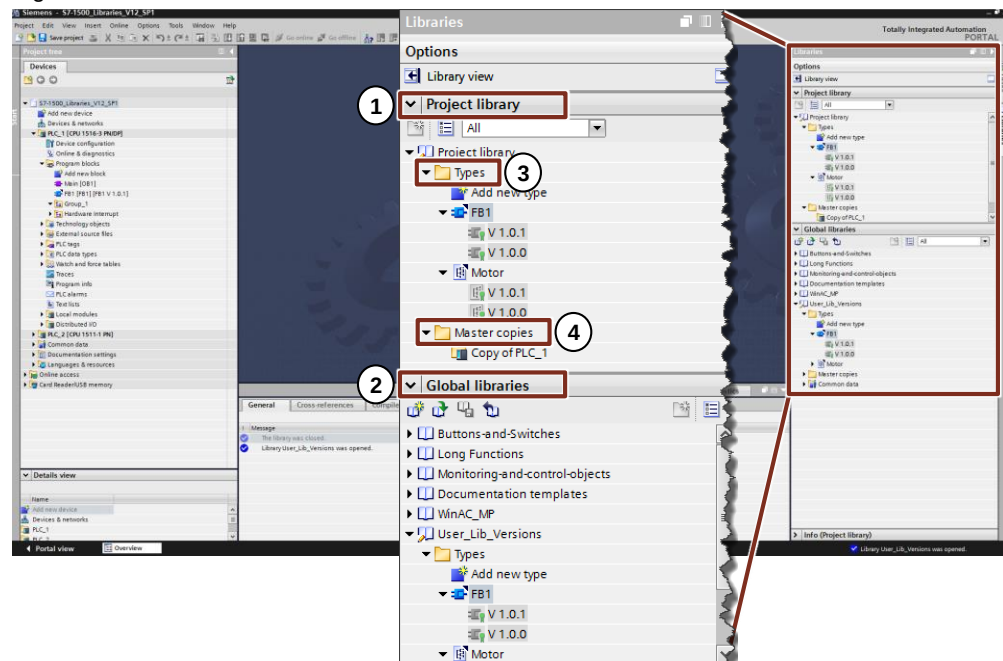
All descriptions and step-by-step instructions in this guide, as well as system behavior, pertain to TIA Portal.

SIMATIC S7-1200 and S7-1500 PLCs

This guide pertains to the SIMATIC PLCs of the S7-1200 and S7-1500 product families.

1.2 Libraries and library elements in TIA Portal

Figure 1-2: Libraries in TIA Portal



1. "Project library"

- This library is integrated in the project and is managed with the project
- It allows reuse within the project

2. "Global library"

- Standalone library
- Can be used in multiple projects
- For distribution of finished library versions to other users

A library contains two different ways of storing library elements:

3. "Master copies"

- Copies of configuration elements in the library (e.g. blocks, hardware, PLC tag tables, etc.)
- Copies are not linked to the elements in the project.
- Master copies can consist of multiple configuration elements.

4. "Types"

- Types are linked to their instances in the project. when types are modified, their instances in the project are updated; this is supported by the system.
- Supported types are blocks (FCs, FBs), PLC data types, HMI screens, HMI faceplates, HMI UDTs, scripts and more. See chapter [8.2 Overview of all library elements](#) and chapter [8.3 Differences in types between PLC and HMI](#).
- Subordinate control elements are automatically typified.
- Types are versioned: Changes can be made by creating a new version.

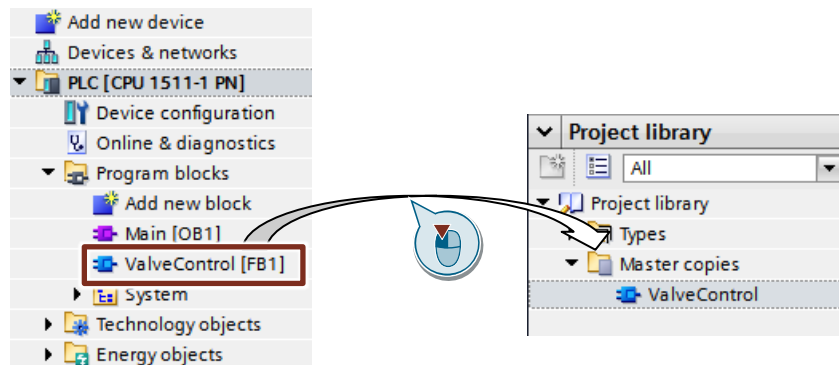
1.3 Working with types and master copies

On the basis of PLC blocks, this chapter will describe the basics of working with types and master copies in project libraries and global libraries.

1.3.1 Storing a block as master copy in project library

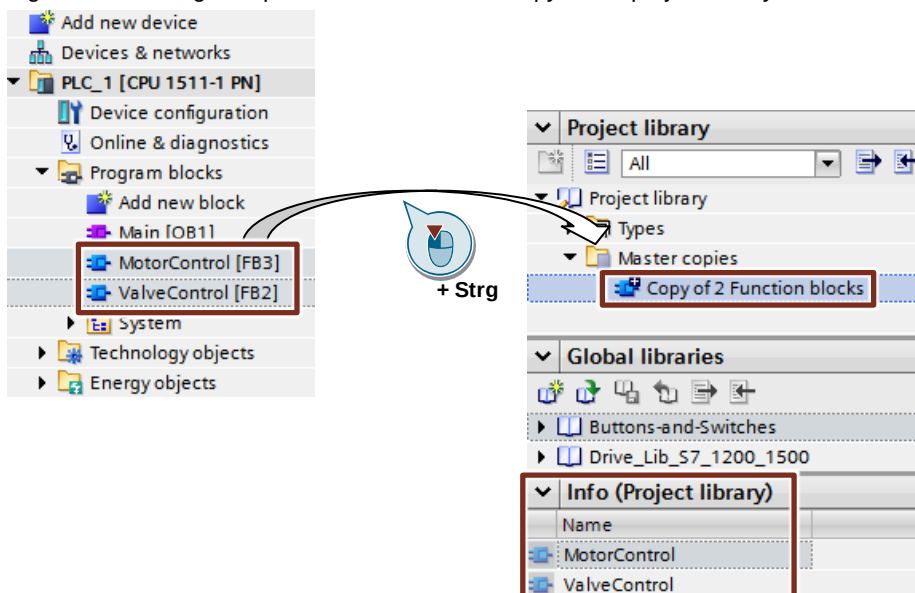
Blocks can be stored with drag & drop as loose copies under "Master copies" in the "Project library".

Figure 1-3: Storing a block as master copy in project library



If you wish to store multiple blocks as a master copy in the project library, you can store the blocks individually or as a group. Press "Ctrl" or "Alt" when dragging and dropping to store multiple blocks as a group. You can give the group any name you like. The contents of the group appear under "Info". You can copy the complete folder structure to the Master copies by dragging and dropping with "Ctrl + Shift".

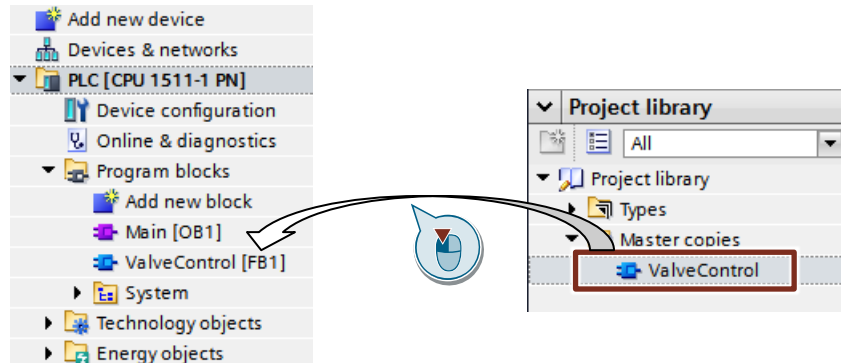
Figure 1-4: Storing multiple blocks as a master copy in the project library



1.3.2 Adding a block to project as master copy

The "Project library" is used as a storage location for reusable elements (e.g. blocks) during the project. The blocks can be added to the device with drag & drop.

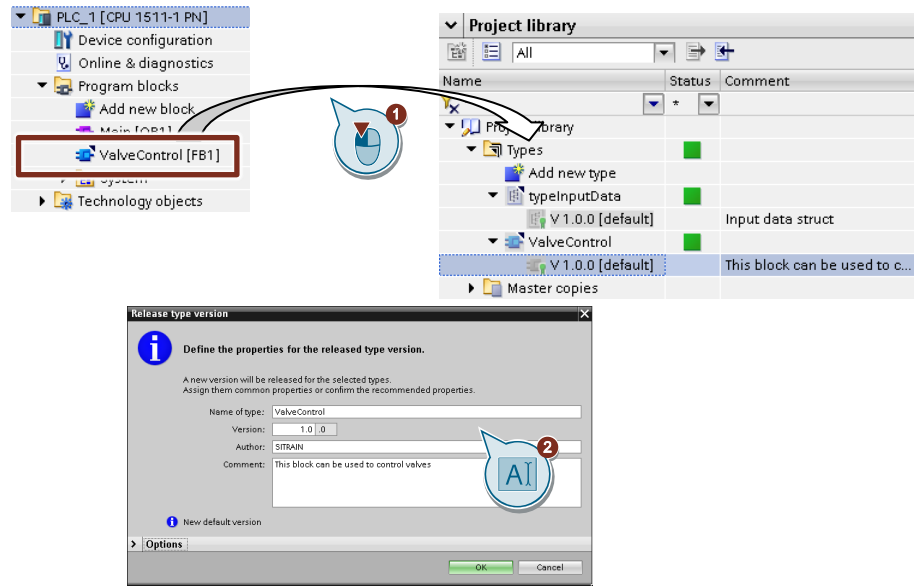
Figure 1-5: Adding a block to project as master copy



1.3.3 Storing a block as a type in the project library

Blocks are versionable and can be stored via drag & drop under "Types" in the "project library". A type possesses metadata (name, version, author, comments), which can be edited when the type is created.

Figure 1-6: Storing a block as a type in the project library



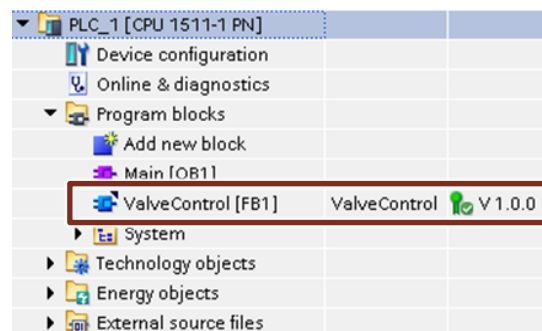
Note

In order to typify safety blocks, they must be called in the F sequence program. The F program must be consistent.

Result

All typified blocks are marked in the program as a type instance with a triangle in the object icon, type name, and version.

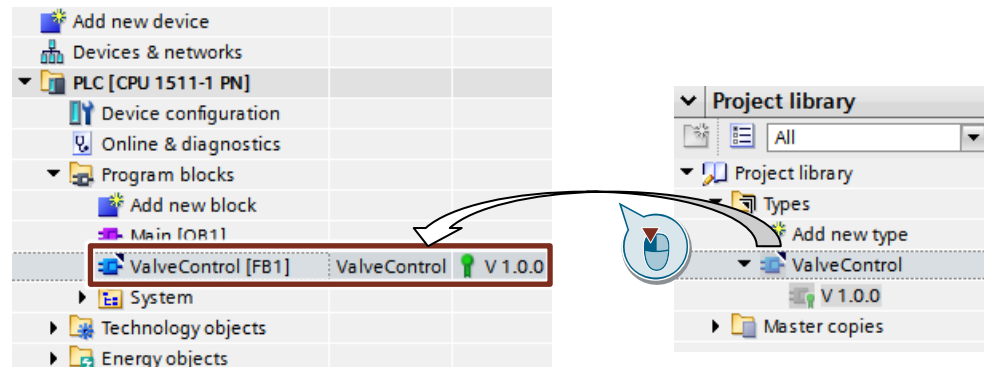
Figure 1-7: Type instances in the program



1.3.4 Using typified blocks in a project

The project library is used within the project as a storage location for reusable elements (e.g. blocks). The typified blocks can be instantiated in the program with drag & drop.

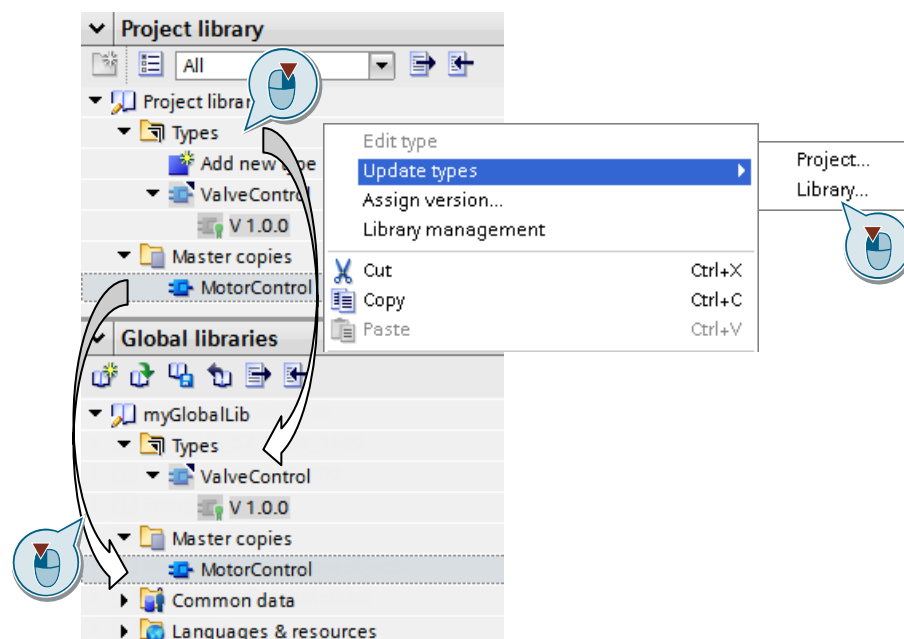
Figure 1-8: Instantiating a block as a type in a project



1.3.5 Copying blocks into global library as master copy / type

- All elements of a project library can also be stored in a global library.
- Master copies can be stored directly in the global library via drag & drop, as described above.
- Types must always be created in the project library before they can be stored in the global library. Types are updated in a global library with the function "Update types > Library".
Note: Drag & drop copies the new type versions without modifying the default version in the global library.

Figure 1-9: Copying master copies / types from project library into the global library



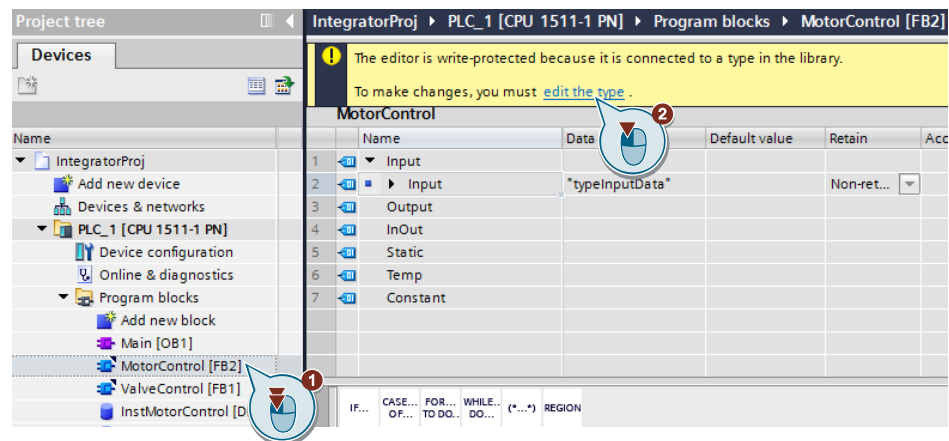
1.3.6 Editing a type

Note Types being edited always require a test environment. In other words, the type must be instantiated in a PLC program.

Editing and releasing a single type

When you open typified elements via the project tree, they will be write-protected in the editor. You can change the type to the [in test] status to edit it.

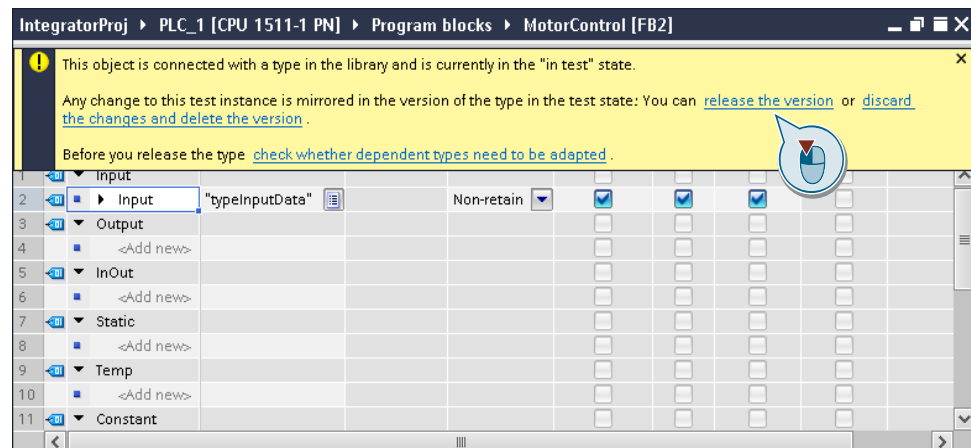
Figure 1-10: Editing a type



The editor function bar is located behind the yellow info pane. Click the exclamation mark to close or reopen the info pane.

When you have finished editing, you can either approve the type or discard the changes and delete the version. Before you approve the type, you can check whether dependent types have to be modified.

Figure 1-11: Releasing a type



Note In order to typify safety blocks, they must be called in the F sequence program. The F program must be consistent.

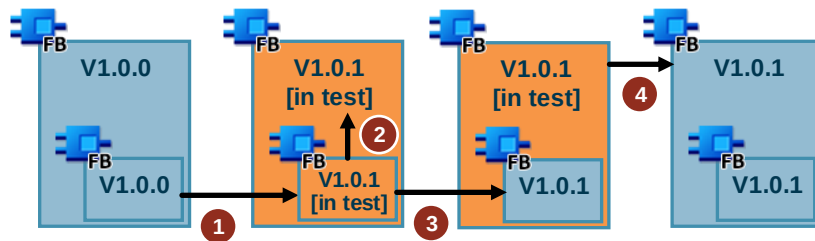
Editing and releasing nested types

If you have multiple types nested within each other, these types will directly impact each other. So that only the current versions of all types are used at all times, you should edit a subordinate type and approve it. Then you may need to update the higher-level types.

Note

Dependent types will have to be modified in the event of structural changes, for example to the block interface.

Figure 1-12: Editing and releasing nested types



1. Switch the block that you wish to edit to [in test].
2. Release the subordinate block and assign it a new version. Enable the option "Set dependent types to edit mode" to modify any dependent types as necessary. This option will be enabled by default if changes are incompatible with one another.
Alternatively, before releasing the version, you can check whether dependent types have to be modified and then switch these to [in test].
3. Release the superordinate block and assign a new version.

Note

You also have the option of releasing all types that are [in test] with the command "Release all versions".

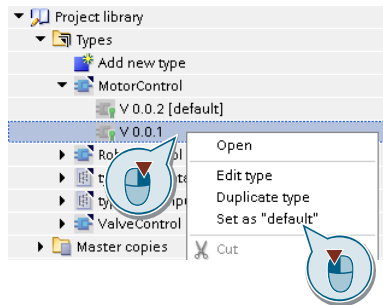
1.3.7 Handling with default version

If you release or update versions, the highest approved (released) version will be used as "Default". The default version will be used when executing library functions, for example when updating or exporting library texts.

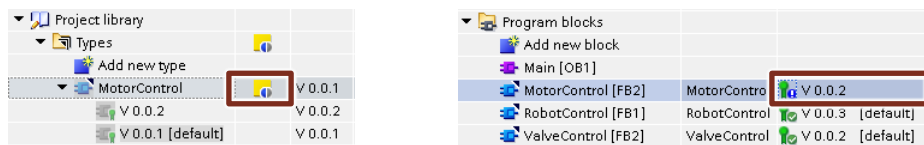
Alternatively, you can define a different approved version as the "Default". This allows you to push older versions to your project.

To do this, proceed as follows:

1. Select the version that you wish to set as "Default".
2. Right-click on the version and select "Set as "default"".



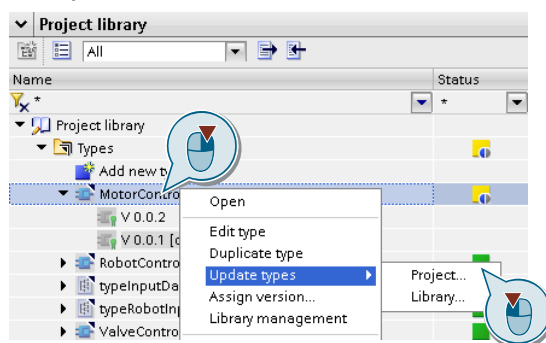
3. From the status of the library block in the project library, as well as from the project tree, it is possible to see that the project is not using the "Default" version. If inconsistencies occur, the corresponding status will be displayed.



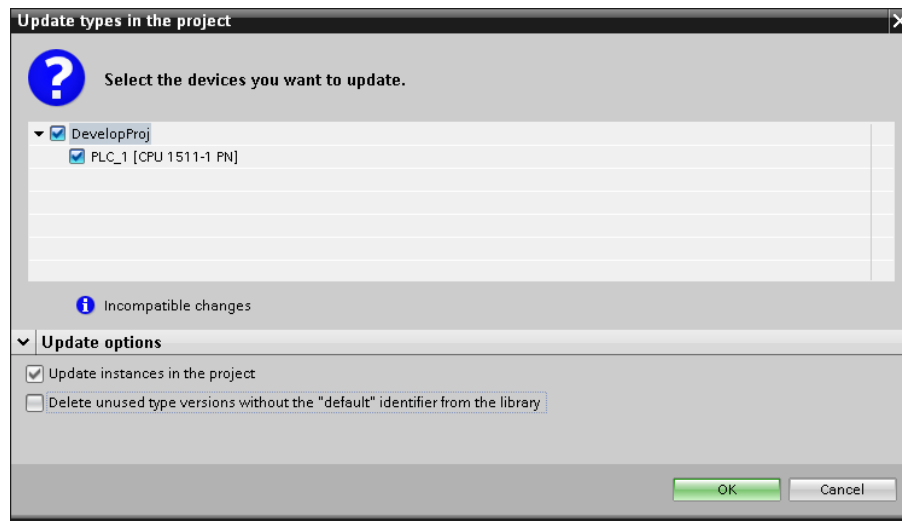
Note

If inconsistencies occur, the corresponding status will be displayed. Recommendation: Resolve the inconsistencies before the project update. (see chapter [7.1.1](#))

4. In order to update the project, right-click on the type and select "Update types > Project".



5. In the dialog, select all PLCs where the modified version must be updated.



Results

The default version will be used instead of the highest approved version during instancing, creation, approval and updating of the type. The default version will continue to be used even if you create newer versions.

An icon in the "Status" column indicates whether a type is consistent or not.

1.3.8 Additional fundamentals

Recommendation

If this is your first time working with libraries in TIA Portal, first read chapter [8 Useful information / fundamentals](#).

1.4 Central storage location for global libraries

Global libraries can be stored on a central server as corporate libraries. These global libraries can be opened automatically when TIA Portal launches. Once newer versions of global libraries are available, you can replace them from a central location. The users of this global library will be informed of this with a notification the next time TIA Portal opens.

Note

For more information, refer to the article "When starting TIA Portal V13 and higher, how do you get a global library to open automatically and use it as corporate library, for example?" at <https://support.industry.siemens.com/cs/ww/en/view/100451450>

and in the TIA Portal Help under "Using libraries > Using global libraries > Using global corporate libraries".

2 Creating a corporate library

Corporate libraries are global libraries that are made available to a user group centrally within a company for use in creating plant projects.

Note

You can find more information on the topic page
"Libraries in the TIA Portal"

<https://support.industry.siemens.com/cs/ww/en/view/109738702>

2.1 Programming guideline

In order to create a corporate library, the authors should adhere to a common and agreed upon programming guideline. This has the following advantages:

- Uniform, consistent programming structure
- Easy to read and understand
- Ease of maintenance and reusability
- Simple and fast error identification and correction
- Efficient work on the same project/library with multiple authors

Note

Recommendation: Programming Style Guide for SIMATIC S7-1200 and S7-1500

<https://support.industry.siemens.com/cs/ww/en/view/81318674>

2.2 Workflow

The workflow described below will show you how to efficiently create, distribute and develop a corporate library. A corporate library is used as a central storage location for TIA Portal elements, which are published for users once they are complete.

This has the following advantages:

- Assurance of an optimized workflow through
 - Process-oriented approach
 - Clear distribution of roles: Integrators, developers, users
- Consistent library throughout the workflow
- Convenient editing and maintainability
- Ease of use in plant projects

The workflow is divided into the following process areas:

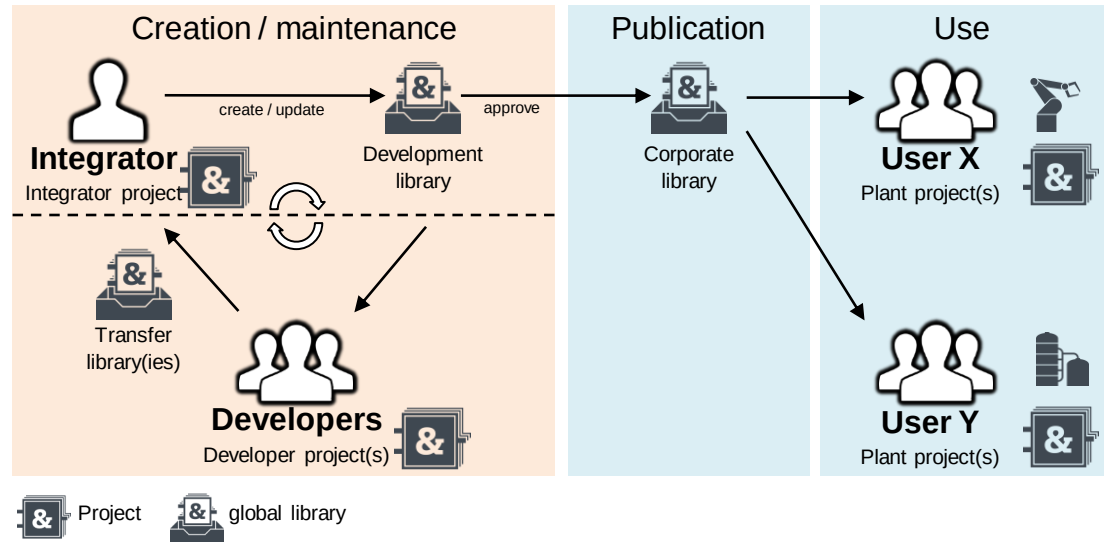
- "Creation / maintenance"
- "Publication"
- "Use"

This sequence serves as the foundation for creating corporate libraries.

2.3 "Creation / maintenance" process

The "Creation / maintenance" process describes how to create and update a "corporate library" (in a developer team). Here, it is necessary to define the various roles, projects and libraries.

Figure 2-1: Workflow



2.3.1 "Integrator" role

The "integrator" heads the developer team. He is the central point of contact for the "developers" and has the task of "creation / maintenance" of the "development library". He merges the supplied "transfer libraries" of the "developers" into an "integrator project". He creates the "development library" from the "integrator project".

2.3.2 "Developer" role

The developer creates new library elements or updates existing library elements. Each developer uses his own "developer project" as the development and test environment. When the developer has finished the library element, he transfers it to the "integrator" in a "transfer library". The "transfer project" contains only the library elements that are to be integrated in the "development library".

Note

If there is only one "developer", he or she will also assume the roles of the "integrator".

2.3.3 "Integrator project" (area of responsibility of integrator)

The "integrator project" serves as the integration environment for the "development library". It has the following properties:

- Contains the total of all type-defined library elements of all developers
- Contains a single call environment for the type-defined library elements
- Can contain older versions of library elements

2.3.4 "Development library" (area of responsibility of integrator)

The "integrator" manages the "development library". It is the basis for all "developers" and can undergo any number of development cycles, which are controlled by the "integrator".

The "development library" has the following properties:

- Global library
- It has an overall version, which is entered in the metadata
- It contains library elements with the following properties:
 - only the current release version from all "developers"
 - metadata (title, comments) are maintained
 - change history (new, modified, deleted library elements)
 - instruction for "developers" if library elements have been deleted

2.3.5 "Developer project(s)" (area of responsibility of developer)

The "developer project" is the test environment of the "developers" for creating or updating library elements. It has the following properties:

- Each developer creates his own "developer project".
- The "developer projects" contain the test environment for the library elements to be created or updated.
- The "developer projects" are not exchanged among the "developers" or with the "integrator".

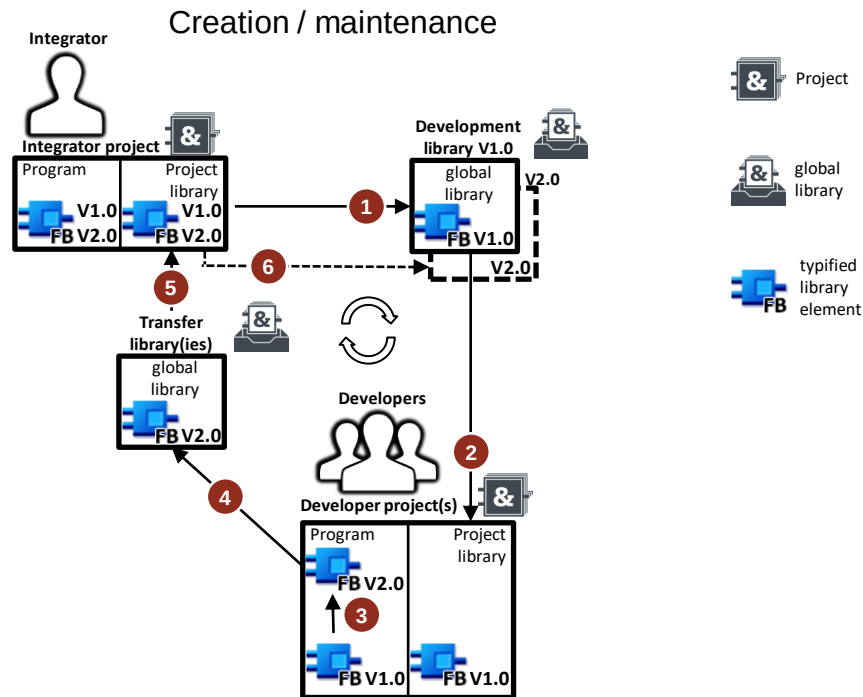
2.3.6 "Transfer library or libraries" (area of responsibility of developer)

The "developer" uses a "transfer library" to transfer the created library elements to the "integrator". It has the following properties:

- Global library
- In the global library, only the objects modified by the developer are selected for update in the "Select for update" column. This reduces integration effort.
- The library element is marked as "[Default]"

2.3.7 "Creation / maintenance" development cycle

Figure 2-2: Development cycle of a library element (here, FB)



1. The "integrator" provides the current status of the "development library" to the "developers" as a global library.
2. The "developers" update their "test projects" with the current "development library".
3. The "developers" create or update library elements. The library elements receive a new version.
4. The "developers" deliver created or updated library elements to the "integrator" in "transfer libraries".
5. The "integrator" integrates all the delivered library elements of the "transfer libraries" into the "integrator project".
6. The "integrator" creates the next version of the "development library" from the "integrator project".

The development cycle can be repeated as often as necessary before the "integrator" performs the "publication" process.

Versioning

When defining versions of the types, the "integrators" and "developers" must agree on a procedure. They must decide who versions the types and how the version numbers will be maintained. Further information can be found in chapter [8.6 Version numbers](#).

Rules for avoiding inconsistencies (area of responsibility of developers)

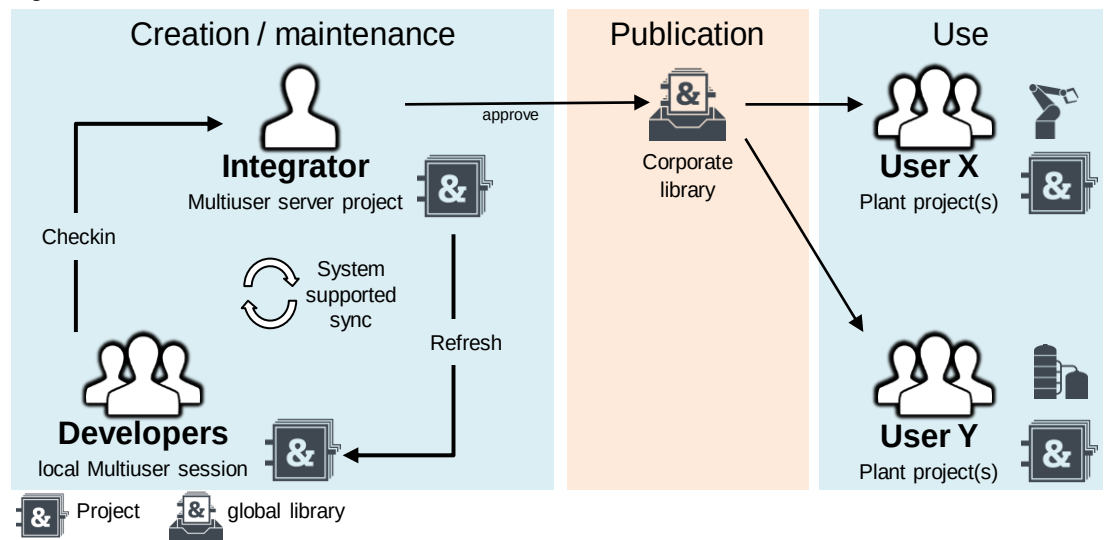
- Multiple developers must not edit the same library elements in parallel.
- The "developers" do not exchange the "transfer libraries" among each other.
- The "developers" do not pass any "developer projects" to the "integrator". They are only edited by the respective "developers".

Rules for avoiding inconsistencies (area of responsibility of integrators)

- The "integrator" determines which "developer" edits which library element.

2.3.8 Alternative: Library development in a Multiuser project

Figure 2-3: Workflow



You also have the option of storing the "integrator project" on the project server. In this way, all developers can work on a common project ("integrator project"), rather than in distributed "developer projects". In this case, each developer works in his/her own session and checks changes into the "integrator project". When the local session is refreshed, changes from other developers are carried over to the local session.

If needed, each developer in the "integrator project" can either create their own PLC or their own software unit where they can develop and test their blocks.

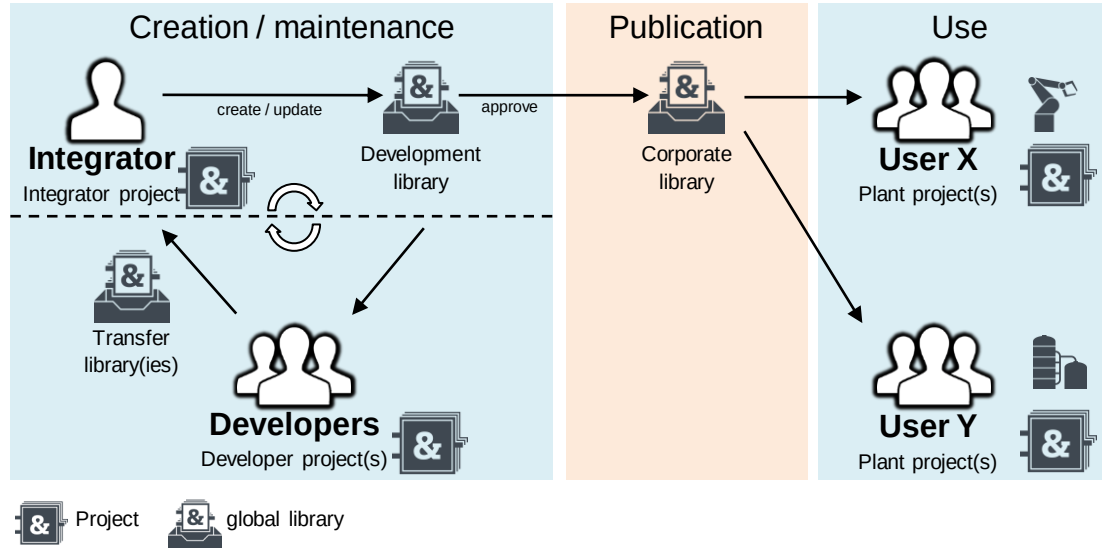
The "integrator project" on the project server offers the following advantages:

- Easy synchronization of changes via Check-in and Refresh instead of cumbersome exchange using "development libraries" and "transfer libraries".
- Developers can respond faster to changes from other developers.
- Markings indicate which types have already been edited. This ensures that types are not modified by multiple developers simultaneously.
- Change history for the library at every check-in to the "integrator project".

2.4 "Publication" process

The "Publication" process describes how the "integrators" publish a "corporate library".

Figure 2-4: Workflow



2.4.1 "Integrator" role

The "corporate library" matches the "development library" in all its properties. The "integrator" sets the point in time when the "Creation / maintenance" process is complete. The integrator then publishes the "development library" as a "corporate library" for the "users".

2.4.2 Corporate library (area of responsibility of integrator)

The "corporate library" is released for the users. It has the following properties:

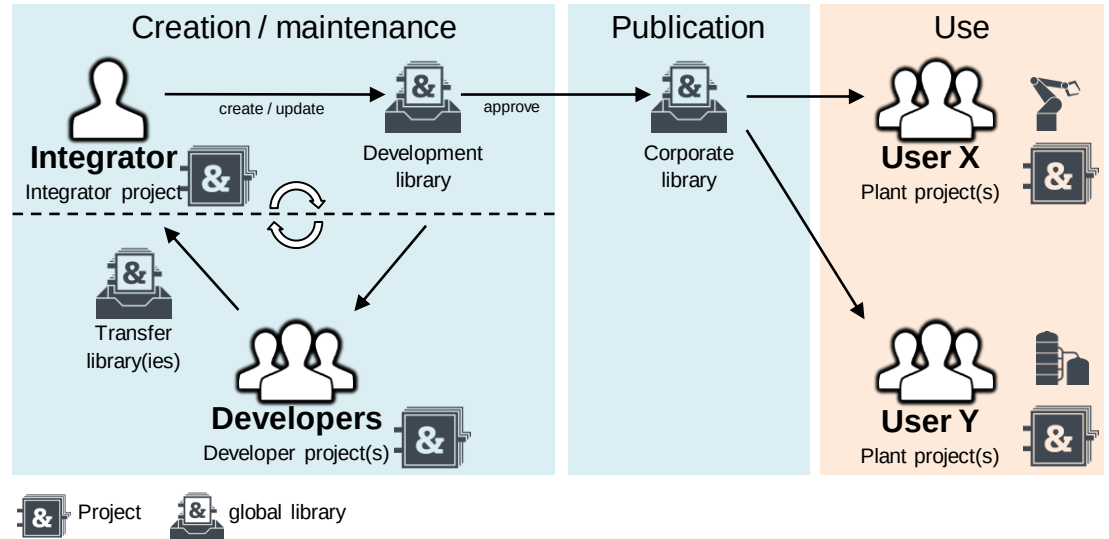
- Copy of the "development library" that the "integrator" declares as the "corporate library".
- The metadata (author, date, version, comment, etc.) of the "corporate library" are maintained.
- It is published as an archived file (e.g. zal17, zip).
- The "integrator" archives all published "corporate libraries" for later traceability.
- The publication is accompanied by documentation with the following contents:
 - Block description (function of the block, interface description, etc.)
 - change history (new, modified, deleted library elements)
 - Instruction for "users" if library elements have been deleted
- The "integrator" creates the documentation with an editable data format (e.g. docx file).

- The "integrator" publishes the documentation in a non-editable format (e.g. PDF file).
- TIA Portal offers the ability to open user-defined documentation. You can create descriptions of your library elements which will then be displayed.
For more information, launch the TIA Portal Help and open the chapter entitled "Editing project data > Providing user-defined documentation".
- The library has been tested for the corresponding target systems.
- The library has write protection / know-how protection if needed.

2.5 "Use" process

The "Use" process describes how to use a "corporate library".

Figure 2-5: Workflow



2.5.1 "User" role

"Users" are people who employ a "corporate library" only in their "plant projects". They do not modify library elements or the "corporate library".

2.5.2 Plant projects (area of responsibility of user)

The "plant project" is a specific project for a machine. It has the following properties:

- Contains library elements of "corporate library"
- Contains plant-specific elements not in the corporate library
- To maintain a good overview, use only one version of a library element and delete unused versions of the library elements.

3 Applications in TIA Portal for the "integrator"

3.1 Working with the "integrator project"

Observe the following information when working on the "integrator project":

Note If the library elements for SIMATIC S7-1200 and S7-1500 PLCs need to be available, always use an S7-1200 PLC to create them. Also be aware of the properties of know-how-protected blocks in chapter [8.7 Know-how protection](#).

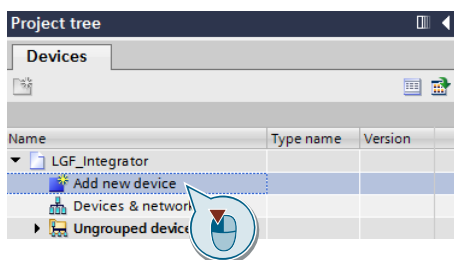
Note If you wish to use the library elements with PLCSIM Advanced, enable the parameter "Support simulation during block compilation" for the project.

Detailed information can be found in the following application example in chapter 2.1, Configuration in TIA Portal.

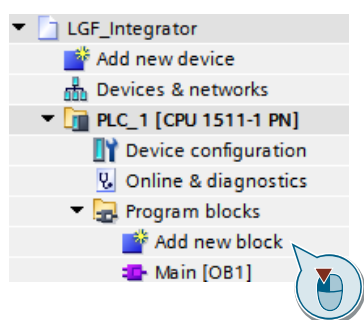
<https://support.industry.siemens.com/cs/ww/en/view/109739660>

3.1.1 Adding new library elements from "transfer libraries" to the "integration project"

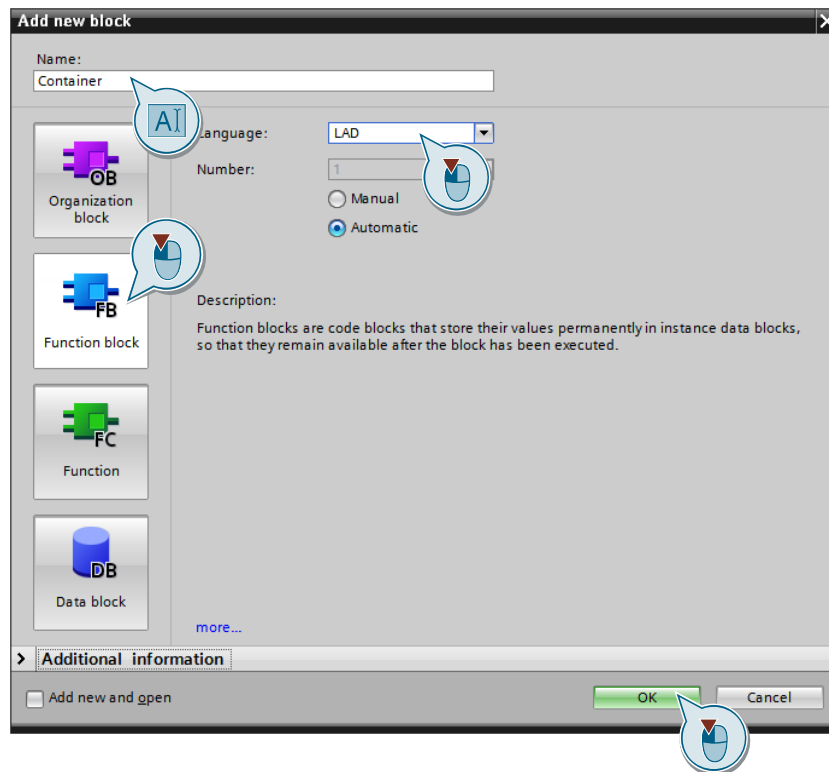
1. If you do not yet have an "integration project", create a new TIA Portal project.
2. Add a device (SIMATIC S7-1200 or S7-1500 PLC) with "Add new device".



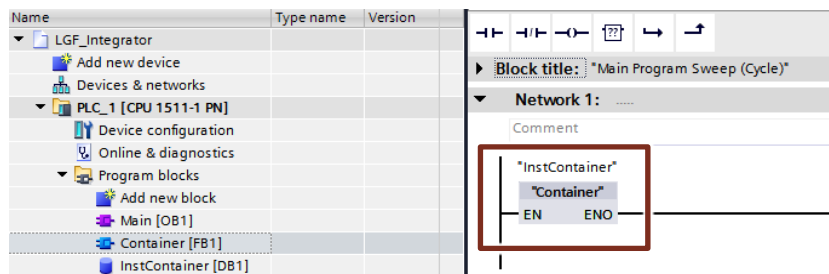
3. Create a new block with "Add new block".



4. Give the FB the name "Container". Use the programming language "LAD" or "FBD". Confirm the dialog with "OK".



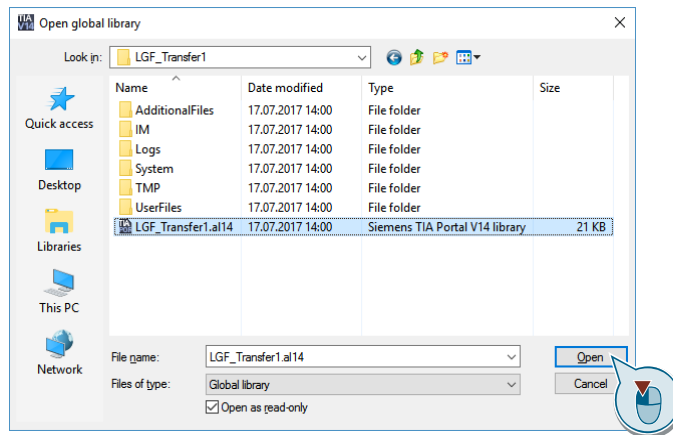
5. Call the FB "Container" in OB "Main" and give the associated instance data block the name "InstContainer".



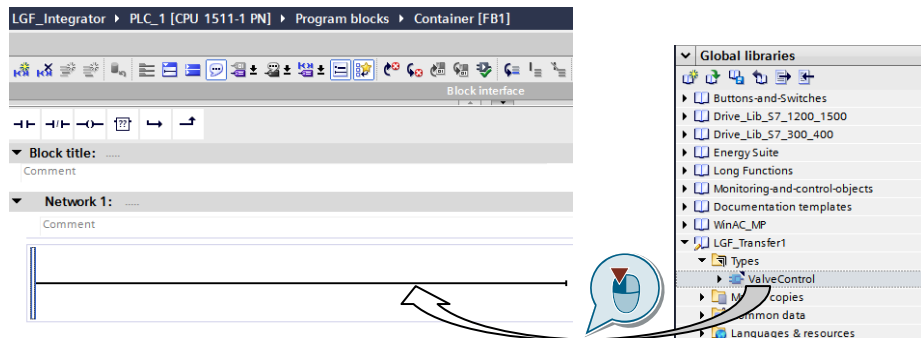
6. Open the FB "Container" with a double-click on the block in OB "Main".

3 Applications in TIA Portal for the "integrator"

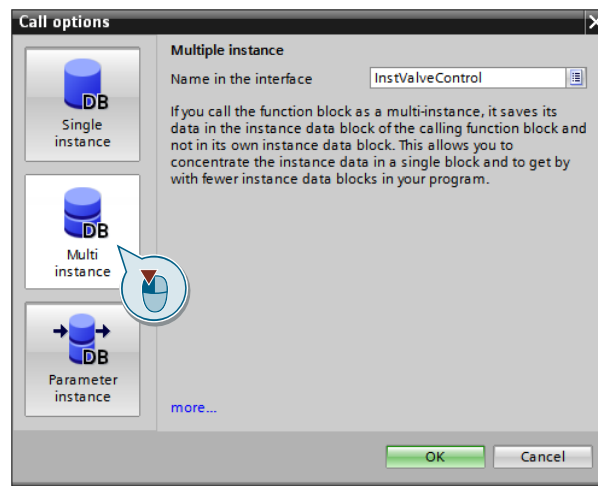
7. Open the "transfer library" supplied by the "developer" (here: "LGF_Transfer1").



8. Add the typified blocks from the "Types" folder to the FB "Container". For a better overview, add each block to its own network.



9. Define the call of the FB as Multiple instance.



10. For FCs, create dummy tags in the temporary or static range of the "Container" block. Connect these with the inputs and outputs of the FC that is called.

Result

A new type from the "transfer library" has been integrated in the project library of the "integrator project". All typified blocks are called in the FB "Container".

Repeat the process until all types are instantiated in the project. If new types are added at a later point in time, you will have to instantiate them in the same way.

3.1.2 Updating existing library elements from the "transfer libraries" in the "integrator project"

Preparation

Check the type versions in the "transfer library" and project library in the "integrator project".

1. Open the "integrator project".
2. Open the transfer library delivered by the developer with the updated library elements (here: TransferLib). You can see objects requiring an update in the "Select for updating" column.
3. Navigate to the project library of your "integrator project" and compare the versions of the types. If needed, you can also compare individual blocks with the quick compare function.

Project library						
 	All	 				
Name	Status	Version	Last change	Author	Comment	
 Project library	*					
 Types						
 Add new type						
 MotorControl		V 0.0.1				
 V 0.0.1 [default]		V 0.0.1	7/29/2021...	Siemens	The block can be used to control m...	
 typeInputData		V 0.0.1				
 V 0.0.1 [default]		V 0.0.1	7/29/2021...	Siemens	Input data type	
 ValveControl		V 0.0.1				
 V 0.0.1 [default]		V 0.0.1	7/29/2021...	Siemens	The block can be used to control va...	
 Master copies						
						
Global libraries						
 	All	 				
Name	Status	Select for updating	Version	Last change	Author	Comment
 TransferLib						
 Types						
 typeInputData			V 0.0.2			
 V 0.0.2 [default]			V 0.0.2	7/29/2021 ...	Siemens	Extension
 Master copies						
 Common data						
 Languages & resources						

Note

If you want to check a project library for differences to a global library, you can use the TIA Openness Library Compare tool:

<https://support.industry.siemens.com/cs/ww/en/view/109749141>

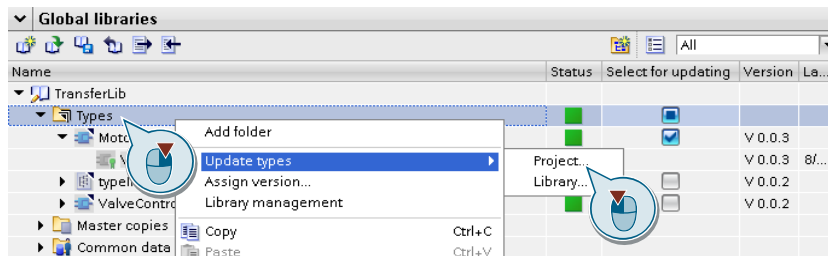
Result

- If the versions of the types in the "transfer library" are **higher**, integrate the library elements into your "integrator project".
- If the versions of the types in the "transfer library" are **lower** than the versions in the "integrator project":
Consult with the "developer" to find out how this happened. The "developer" evidently has not worked with the latest version of the "development library".

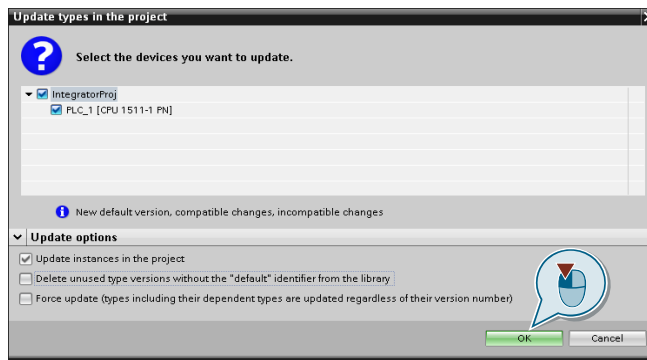
Updating existing library elements from the "transfer libraries" into the "integrator project"

The following step-by-step instructions demonstrate the integration process using the example of a subordinate type.

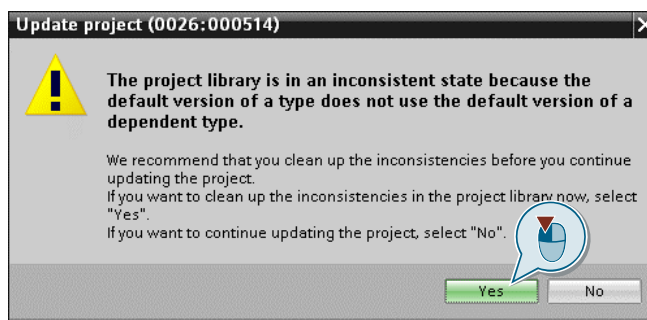
1. In order to integrate the updated types from the global "transfer library" into the "integrator project", right-click on the "Types" folder in the global "TransferLib" and select "Update types > Project".



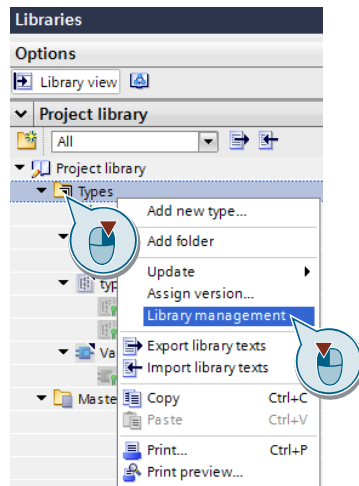
2. Confirm the dialog with "OK". The unused types can be deleted in the project library.



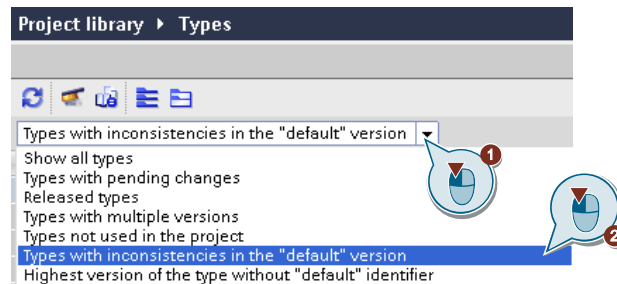
3. The following message will **only** appear if inconsistent changes were made to objects. Press "Yes" to first resolve the inconsistencies in the project library. The project will not be updated at this point. If you press "No", the project would be updated.



- Right-click on "Types" in the project library and select "Library management".



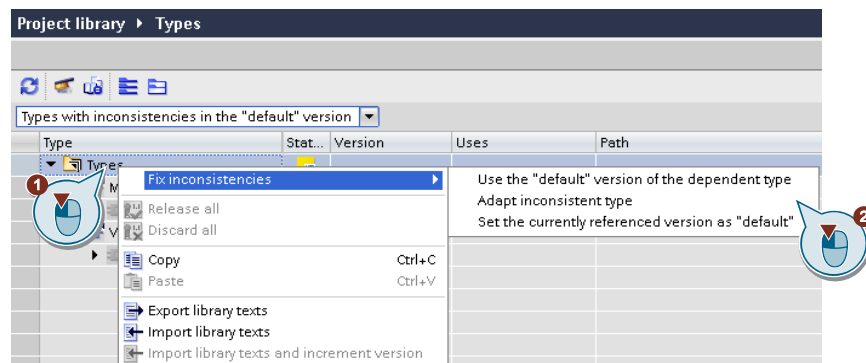
- Use the filter dialog to check whether the types you wish to update have higher-level types. To do this, select "Types with inconsistencies in the "default" version".



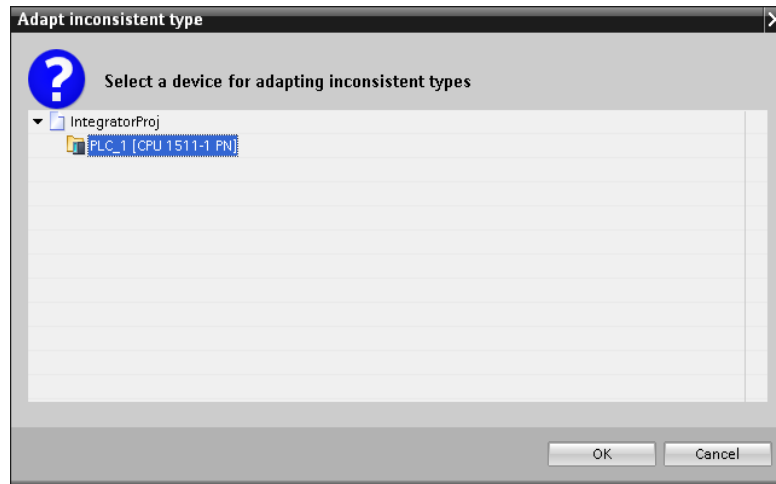
The complete filter dialog is described in chapter [8.5.2 Filter dialog in the Library management](#).

All types will be displayed which have a direct or indirect dependency with the type "typeInputDate". If no types with dependencies appear here, the integration is complete. You can exit the instructions at this point.

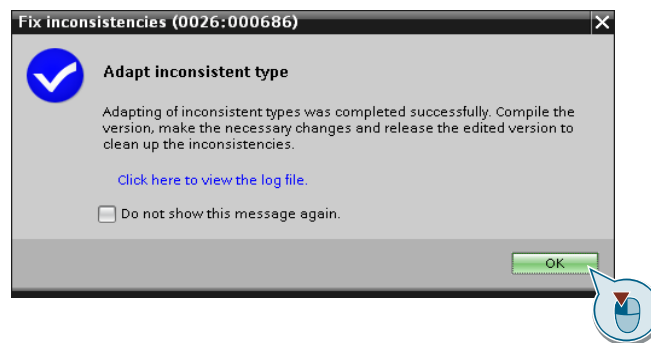
- Right-click on the "Types" folder in the Library management. Select "Fix inconsistencies > Adapt inconsistent type".



7. Select "PLC_1" as the device for the adaptation of the inconsistent type and then click "OK".

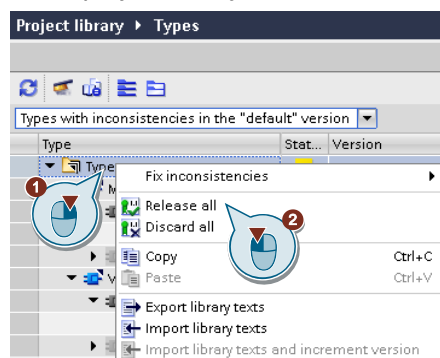


8. Confirm the dialog with "OK".

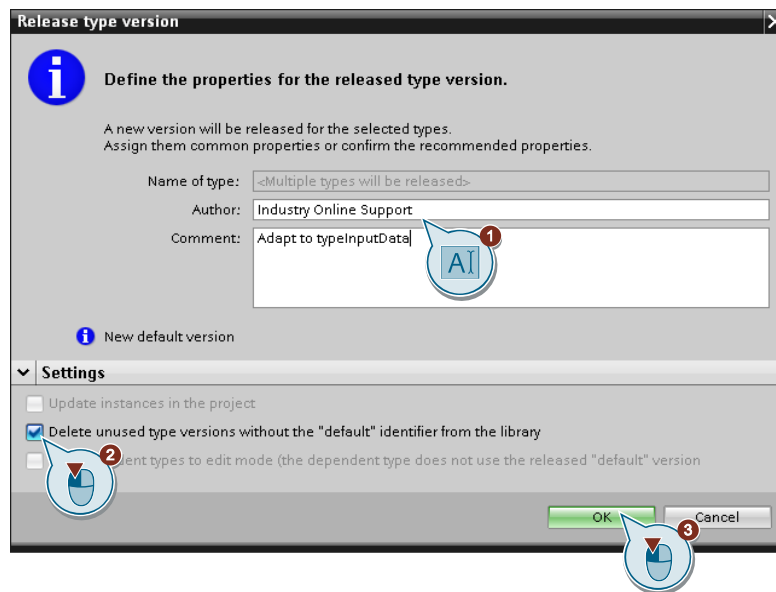


The inconsistent types have been switched to [in test]. You now have the ability to adapt your types in the project.

9. In order to approve all types in the [in test] status, right-click the "Types" folder in the project library and select "Release all".



10. Update the metadata, enable the option ("Delete unused type versions without the "default" identifier from the library" and confirm the dialog with "OK".



Result

The type delivered with the "transfer library" has been integrated into the "integrator project". All superordinate types have been updated. All types with older versions have been deleted.

If necessary, repeat this procedure until all types supplied in the "transfer libraries" have been integrated into the "integrator project" and all dependent types have been updated.

Recommendation

Use the filter dialogs in the Library management to check whether all types have been updated and old types have been deleted. The status of all types is consistent, as can be seen from the green rectangle in the "Status" column.

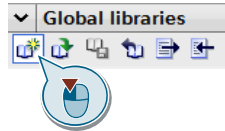
3.1.3 Tips for structuring the container FB

- Structure the dummy tags for each block in a Struct tag for better comprehensibility.
- Give each network the name of the corresponding block that is called.
- Write the metadata (developer, date, etc.) in the network comments.

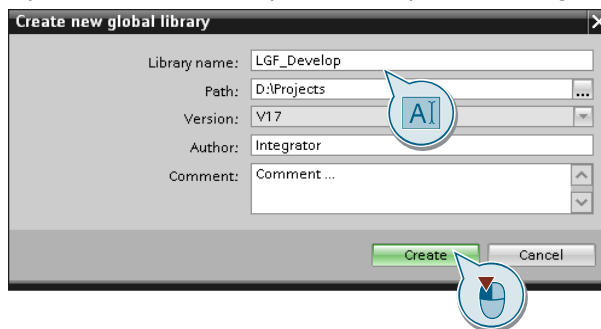
3.2 Working with the global "development library"

3.2.1 Creating the global "development library"

1. Create a global library with the "Create new global library" button.

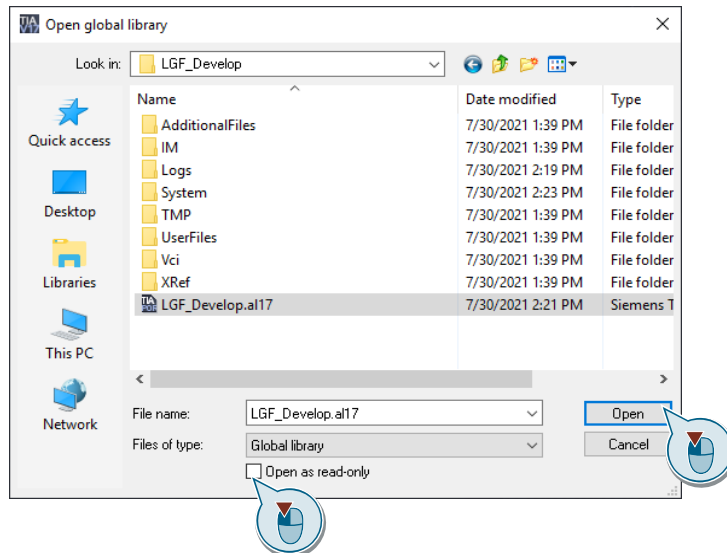


2. Update the metadata (Author, etc.) in the dialog and click "Create".

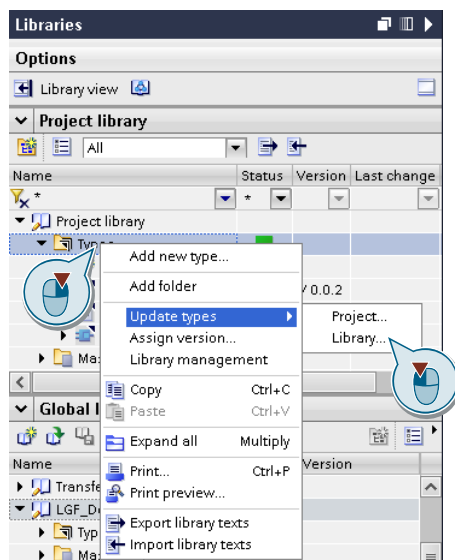


3.2.2 Adding and updating blocks from "integrator project" into the global "development library"

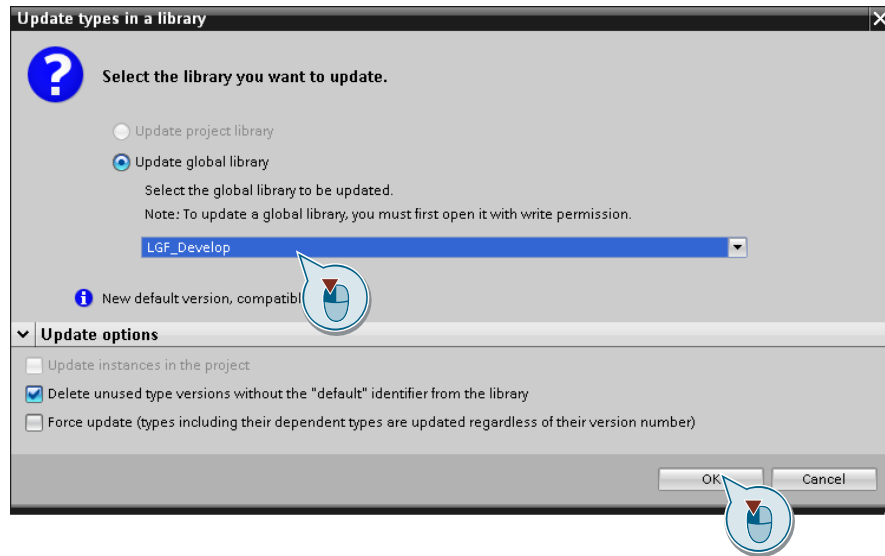
1. Open the "integrator project".
2. Open the global "development library" and uncheck the box "Open as read-only".



3. In order to add the types from the "integrator project" to the "development library", right-click in the project library on the "Types" folder and select "Update types > Library".



4. In the dialog, select "Update global library" and select your "development library". Confirm the dialog with "OK". In the options, you can enable deletion of unused types.



Result

The "development library" contains all types from the "integrator project". All updated types in the "development library" will retain their older versions if you have not enabled the deletion of unused type versions.

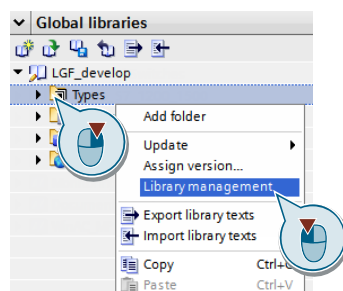
Recommendation

To prevent conflicts and improve manageability in the "development library", remove all older types as described in the following chapter. Every library element should only have one version.

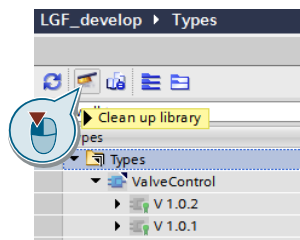
3.2.3 Deleting older versions of types in the global "development library".

Proceed as follows to clean up a global "development library":

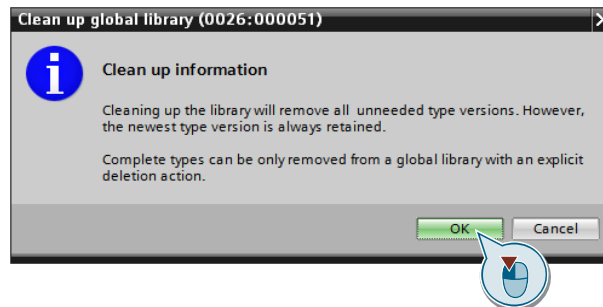
1. Open the "integrator project".
2. Open the global "development library" and uncheck the box "Open as read-only".
3. In the "development library", right-click on the "Types" folder and select "Library management".



4. To keep just the current versions, click "Clean up library".



5. Confirm the dialog with "OK".



Result

The "development library" now only contains the current version of the respective types.

You now have a defined version of the "development library". You have two options.

1. Distribute the current version of the "development library" to the "developers" for further editing of library elements.
2. Approve the "development library" as a "corporate library" for the "users".

Recommendation

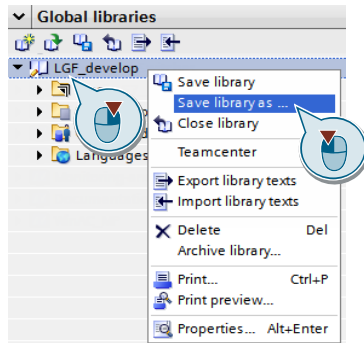
- Maintain the metadata (developer, version, comment, etc.) of the "development library".

3.3 Working with the global "corporate library"

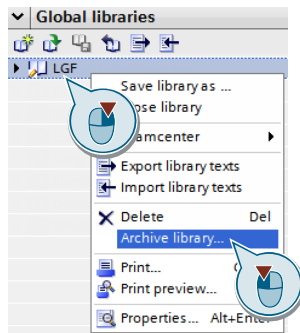
3.3.1 Creating and approving the global "corporate library"

The "integrator" creates a copy of the global "development library" and declares it as a global "corporate library".

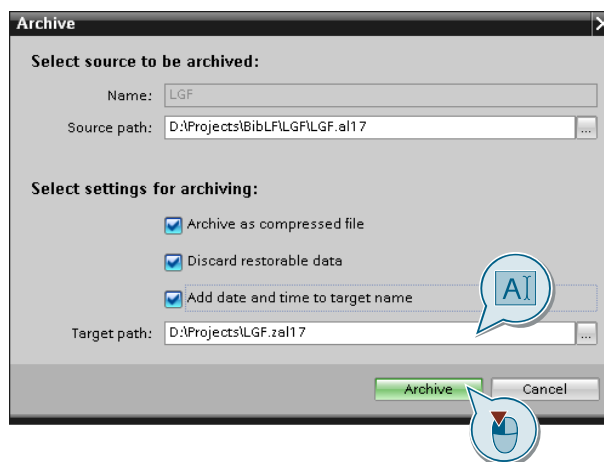
1. Open the global "development library" and uncheck the box "Open as read-only".
2. Right-click on your "development library" and select "Save library as ...".



3. Assign an appropriate name for the "corporate library".
4. Right-click on the global "corporate library" and select "Archive library...".



5. Specify all the options, then define the archive name and storage path. Click "Archive."



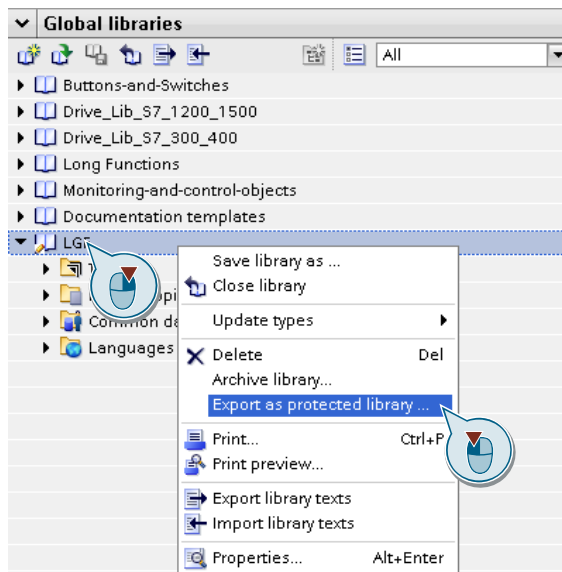
Result

The global "corporate library" now exists as a compressed file and can be published for the "users".

3.3.2 Creating write-protected libraries

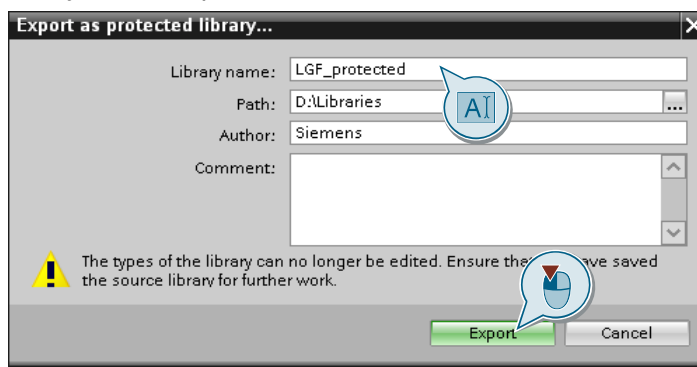
You can protect global libraries by exporting them as protected libraries.

1. Open the global library.
2. Right-click on your global library and select "Export as protected library ...".



The dialog "Export as protected library..." opens.

3. Enter a suitable library name and specify the save directory for the protected library. Click "Export".



Result

The global library is saved in the folder you chose as a protected library and is opened in the "Global libraries" task card.

Properties of write-protected libraries

Write-protected libraries have the following properties:

- Once protected, the write protection on protected global libraries cannot be removed.
- No password is necessary for write protection.
- The global library as well as the types contained within it are protected against modifications.
- When a library is updated from a protected library, the types and type instances in the project are permanently write-protected.

The protected types and type instances of the protected library have the following properties:

- They can be opened in the editor but not modified.
- They cannot be edited.
- Assigning a new version is not possible.
The update mechanism is retained.
- The connection of an instance to a type cannot be removed.

Note

If you want to deliver write-protected types to customers, try to create write-protected libraries with types in a new, higher version, and then deliver these.

If you create write-protected libraries, make sure not to delete your non-write-protected library.

3.3.3 Discontinuation of types

Library blocks, too, are subject to a lifecycle beginning with planning and development and ending with discontinuation. Blocks may cease to be needed, for instance because:

- The function of the block has been replaced by new system functions (newer firmware).
- The functions of a complex block have been divided among multiple other blocks.
- The block type has changed, e.g. FC → FB.

The behavior has been fundamentally changed/reworked, with the result that one knowingly limits the update potential.

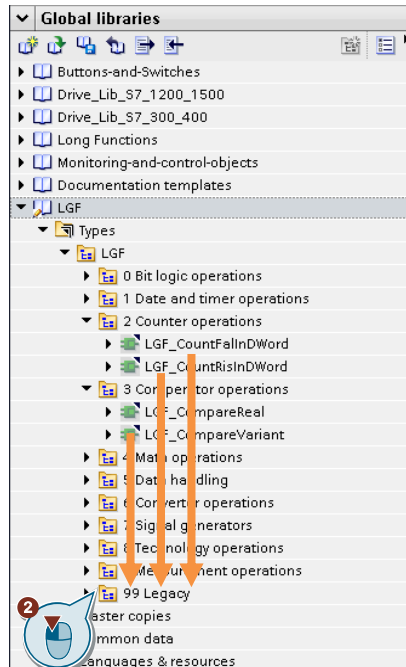
Note

Add the following notes to the blocks and in the library documentation:

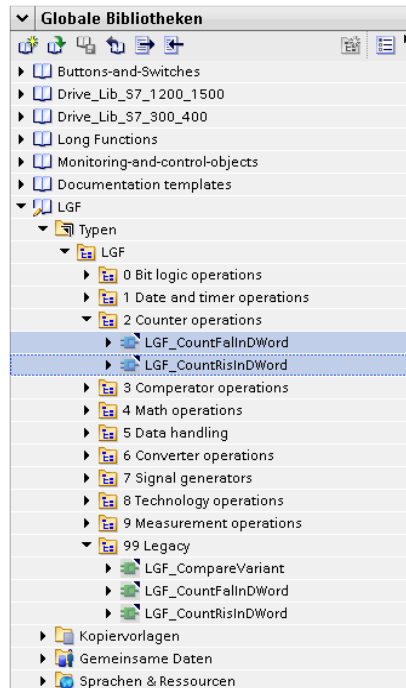
- The block "<block name>" has been moved to the "Legacy" folder.
- The block "<block name>" has been replaced by the "<Block name(s)/system function>" function(s)/system functions

Proceed as follows for blocks that are no longer needed:

1. Open the global library and uncheck the box "Open as read-only".
2. Create a folder in your library with the name "Legacy".
3. Highlight the block or blocks which are no longer needed and move it/them into the "Legacy" folder.



4. If necessary, add the new global library blocks. In the figure below, the FCs "LGF_CountFallInDWord" and "LGF_CountRisInDWord" are replaced with FBs of the same name.



Result

The outdated blocks are saved in the "Legacy" folder of the global library and thus remain in the library. In this way, the user can see that these blocks are no longer current, and can replace them.

Chapter [5.1.5 Replacing discontinued types](#) describes how you can replace discontinued types.

Note

So as not to be continually delivering obsolete types, the types in the "Legacy" folder can be deleted where necessary in subsequent releases of the global library.

4 Applications in TIA Portal for "developers"

4.1 Working with the "developer project"

Observe the following information when working on the "developer project":

Note If the library elements for SIMATIC S7-1200 and S7-1500 PLCs need to be available, always use an S7-1200 PLC to create them.

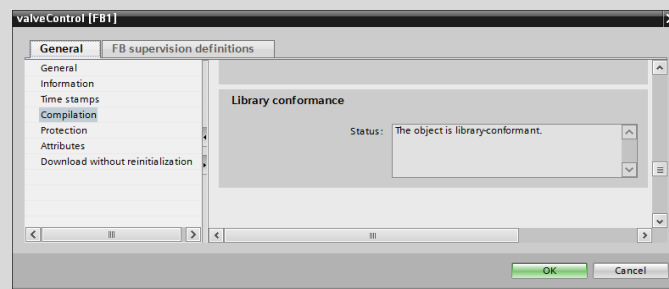
Note If you wish to use the library elements with PLCSIM Advanced, enable the parameter "Support simulation during block compilation" for the project.

You can find detailed information in the following application example in chapter 2.1, Configuration in TIA Portal.

<https://support.industry.siemens.com/cs/ww/en/view/109739660>

Note The block that you wish to create as a type must be library-conformant.
The block contains...

- no global data access or single-instance call of an instance data block.
- no direct access to data blocks, PLC tags or PLC constants.



4.1.1 Updating the "developer project" with the global "development library"

To always develop with the latest blocks, update your "developer project" with the current "development library".

Case 1

You have not made any changes to your types since you handed off the last "transfer library" to the "integrator".

In this case, you can update the new versions of the "development library" in your "developer project". Use the "Force update" option to ensure that the "developer project" matches the state of the "development library".

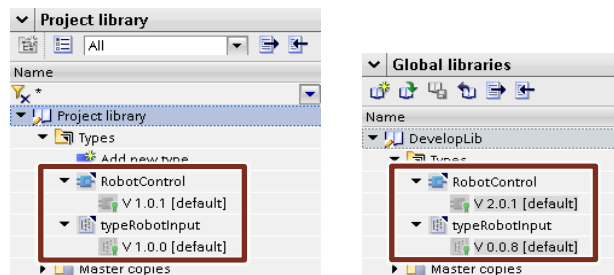
Case 2

You have made changes to types since you handed off the last "transfer library" to the "integrator".

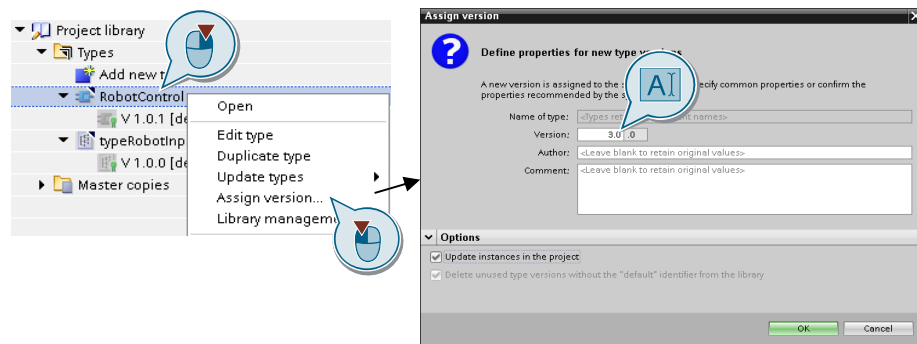
Note

Always avoid this case if possible. It is better to wait with your changes until you have updated your "developer project" with the current "development library".

1. Open your "developer project" and the global "development library".
2. Identify the types that you changed in your "developer project" and in the "development library".



3. Compare the versions of the types from your "developer project", which you changed, with the "development library". If the version number of the type in the "developer project" is lower than the version of the type in the "development library", then assign the type in the "developer project" a higher version number using the function "Assign Version".



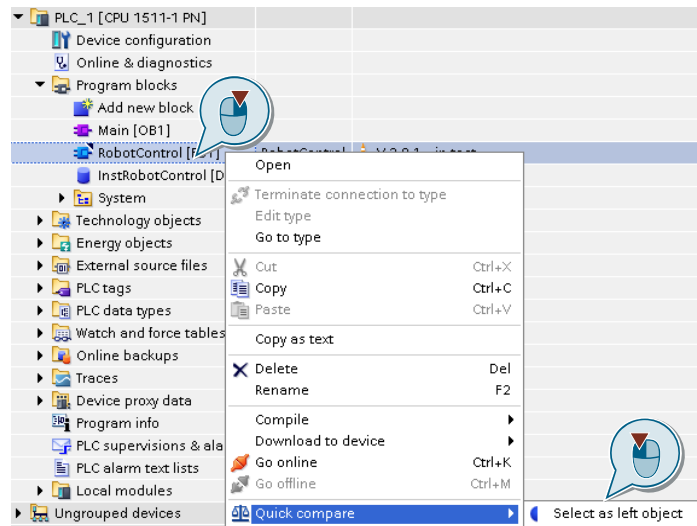
- Update your project with the "development library". To do this, follow the step-by-step instructions in chapter [3.1.2 Updating existing library elements from the "transfer libraries" in the "integrator project"](#) but do not release the types.

Use the following terms analogously:

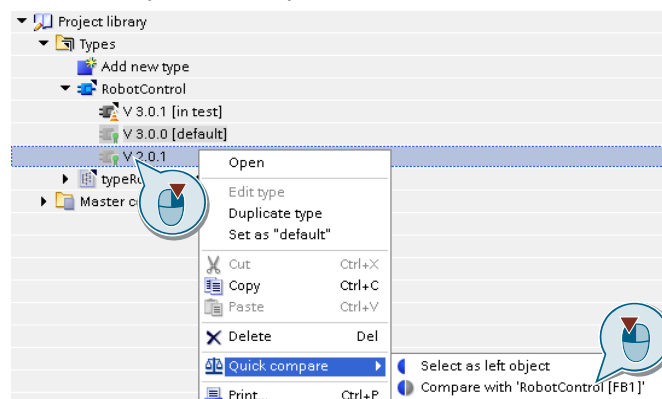
"transfer library" → "development library"

"integrator project" → "developer project"

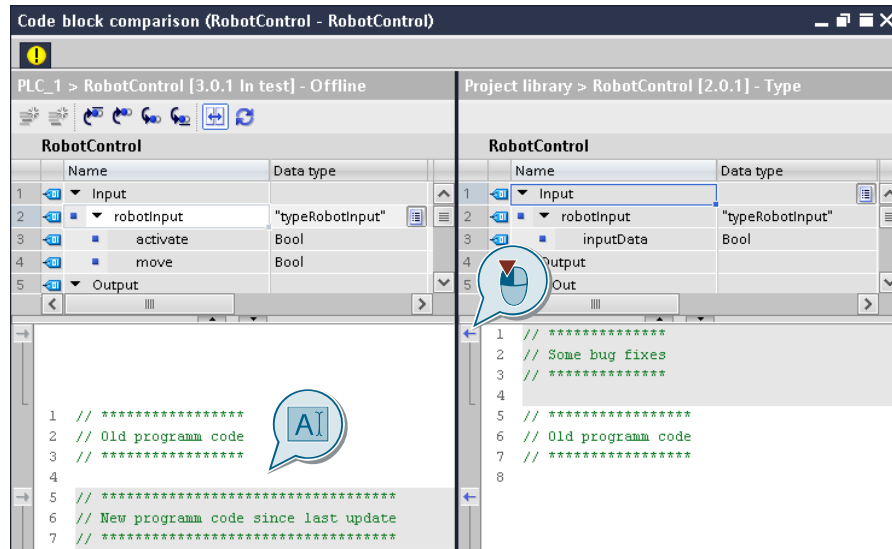
- Now compare the current type versions in the "development project" with the version from the "development library". Right-click on your type and select "Quick compare > Select as left object".



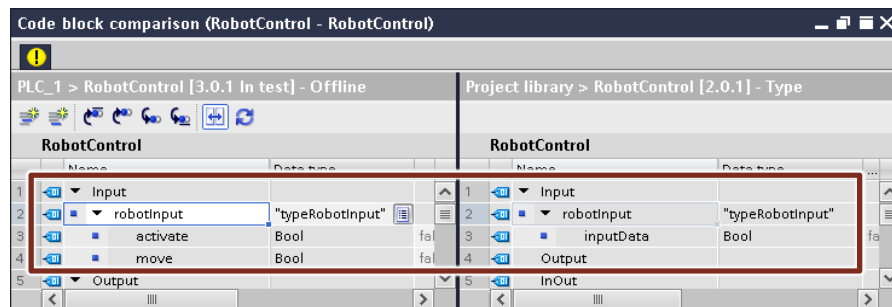
- Right-click the type that you added from the "development library" and press "Quick compare > Compare with...".



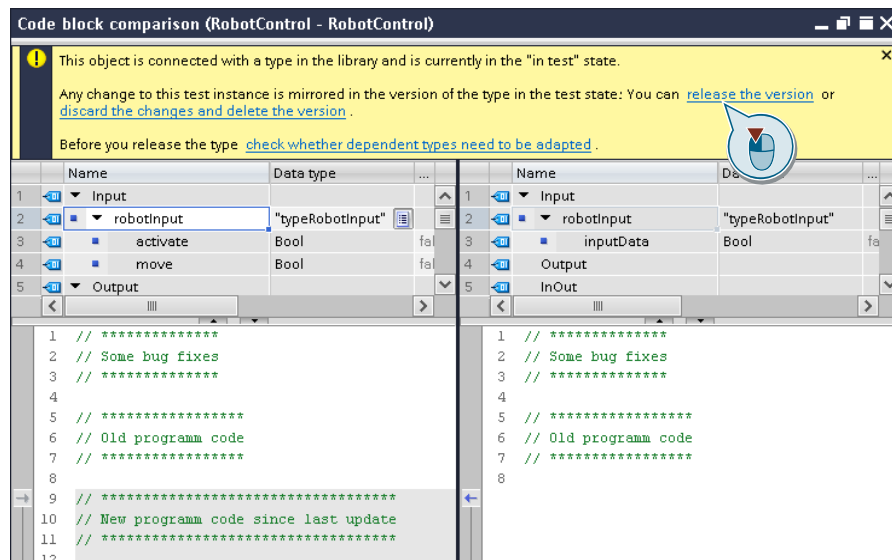
7. Apply the desired changes in the block code.



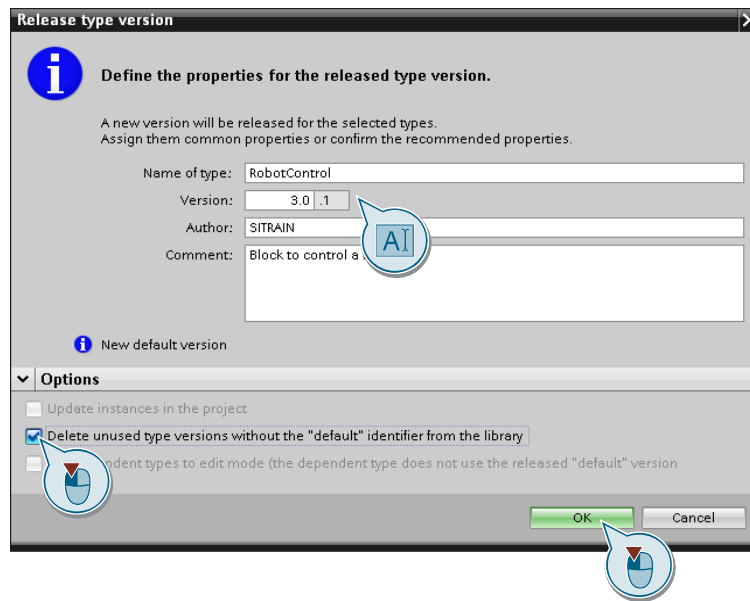
8. Check the changes in the block interface. In this case, the subordinate type (here: "typeRobotInput") in the project has changed.



9. Approve the block with "release the version".



10. Assign a new version and confirm the dialog with "OK".



11. Repeat the steps for all blocks that have been modified.
12. Then delete the older type versions as described in chapter [3.2.3 Deleting older versions of types in the global "development library"](#)..

Note

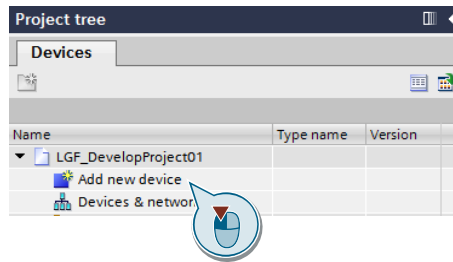
If you want to check a project library for differences to a global library, you can use the TIA Openness Library Compare tool:

<https://support.industry.siemens.com/cs/ww/en/view/109749141>

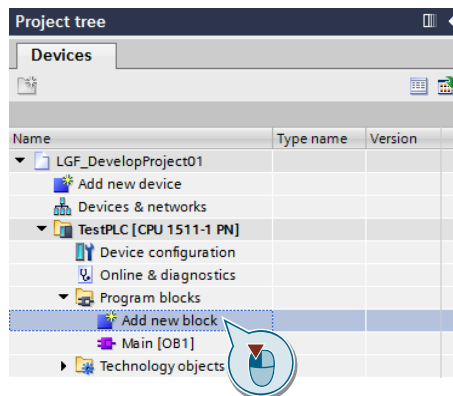
4.1.2 Creating new typified elements

Proceed as follows when you wish to create new typified elements for the "development library".

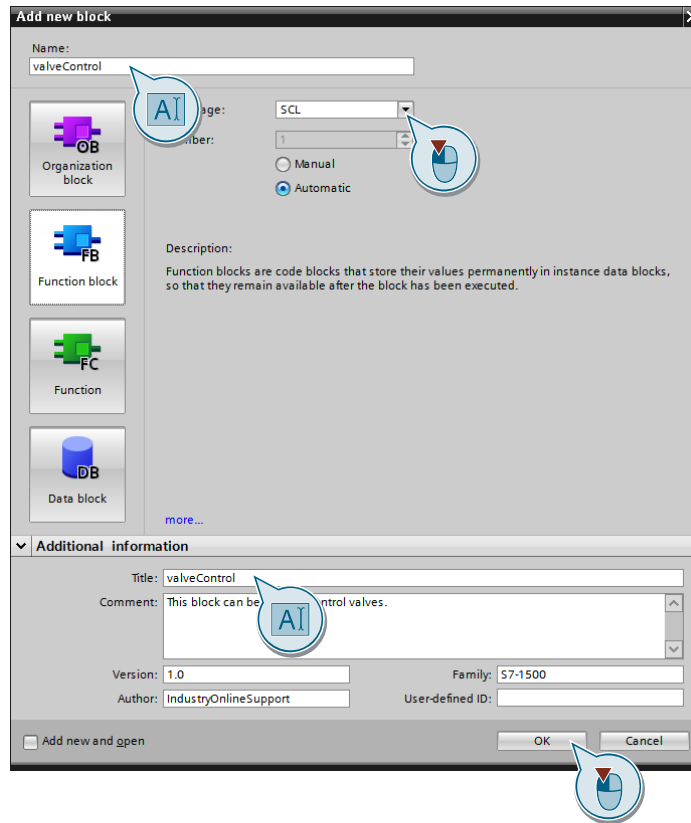
1. Open or create your "developer project".
2. Create a test PLC with "Add new Device". This is the PLC you will use to develop the new block.



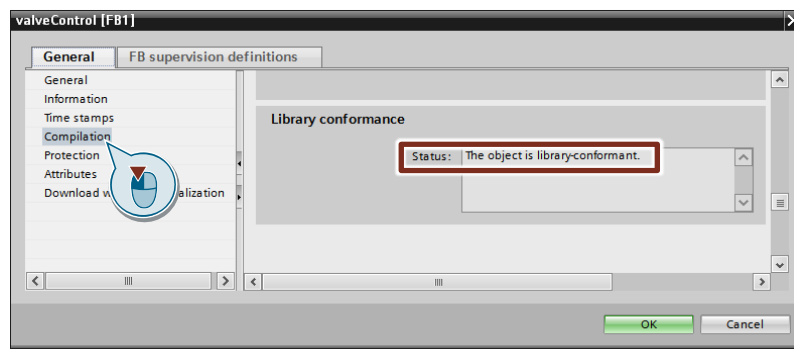
3. Create an FC or an FB with "Add new block".



4. Choose a name, select the programming language and update the metadata in the dialog under "Additional information".

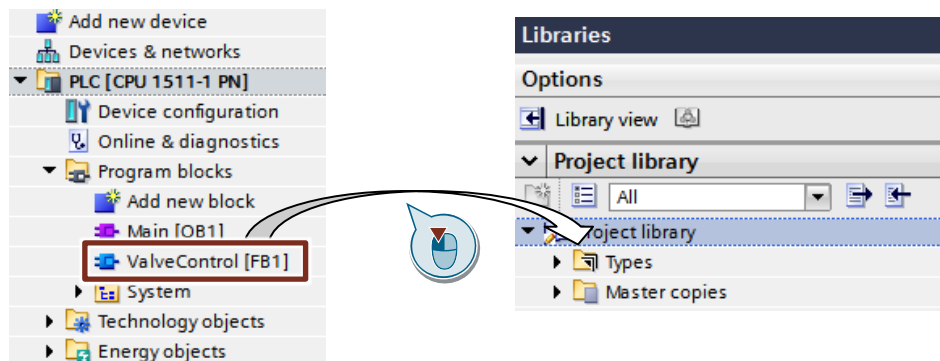


5. Program the block and compile it.
6. Test the block in a suitable test environment.
7. Verify that you have programmed the block such that it is library-conformant. To do this, right-click on the block, open the properties and navigate to "Compilation > Library conformance".

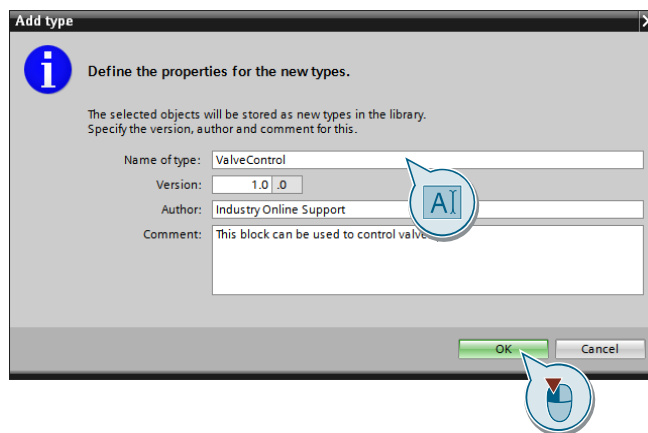


If the block is not library-conformant, check whether the block is modularly programmed. The block must not use any global constants or global tags. Data can only be exchanged via the interfaces of the block.

8. Drag and drop the block into "Types" folder in the project library.



9. In the dialog, define "Name of type", "Version", "Author" and "Comment". Close the dialog by clicking "OK".



Note

Types being edited always require a test environment. In other words, the type must be instantiated in a PLC program.

Result

The block is located in the project library of your "developer project" and is a type with the label "[Default]". If the block has subordinate elements (block call, PLC data types), these will also be stored as a type with an independent version in the project library of your "developer project".

The typified locks and subordinate elements are marked as a type with a black triangle in the object icon.

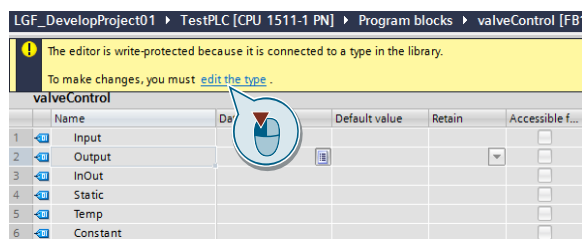
4.1.3 Editing typified elements

Proceed as follows when you wish to edit existing typified elements:

Requirements

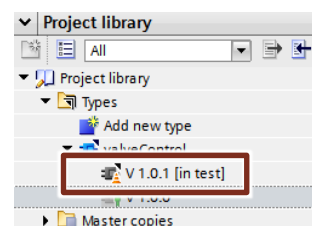
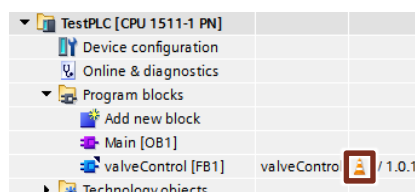
A typified library element is located in your "developer project".

1. Open your "developer project".
2. Open the typified block.
3. Click on "edit the type".



Alternatively, you can right-click in the project tree on the block and press "Edit type".

4. The block is in the [in test] status (see icon on the block in the program and in the project library).

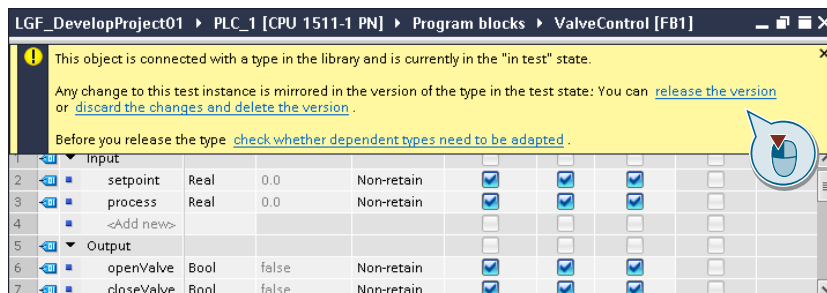


5. Edit the block and keep the [in test] status until it is finished.

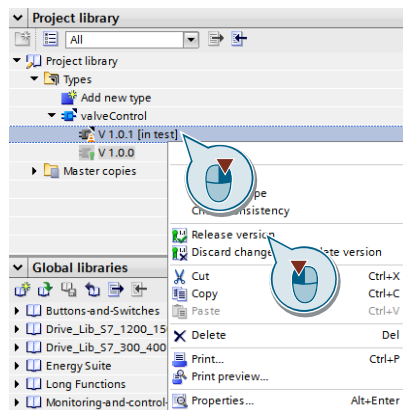
Note

You can load and test blocks that are [in test] status on SIMATIC S7-1200 and S7-1500 PLCs.

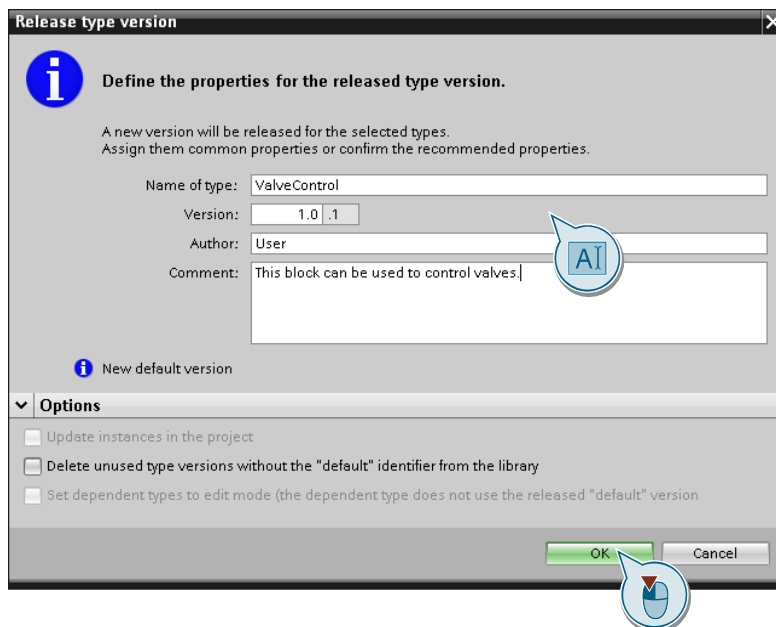
6. When the block is finished, click "release the version".



Alternatively, you can right-click on the type in the project library and press "Release version".



7. Maintain the metadata in the dialog. Define the version. Optionally, choose whether all older type versions in the project library should be deleted. Click on "OK".



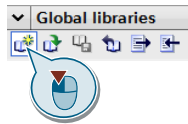
Result

The edited block is located in the project library of your "developer project" and is a type with the label "[Default]". The version is higher than that of the original block.

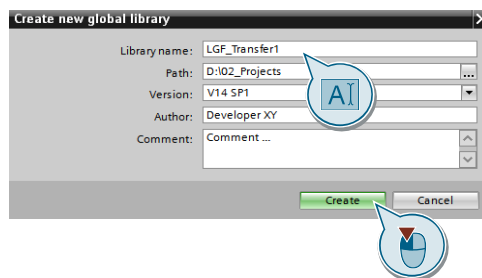
4.2 Working with the global "transfer library"

4.2.1 Creating a global "transfer library"

1. Under "Global libraries", click "Create new global library".

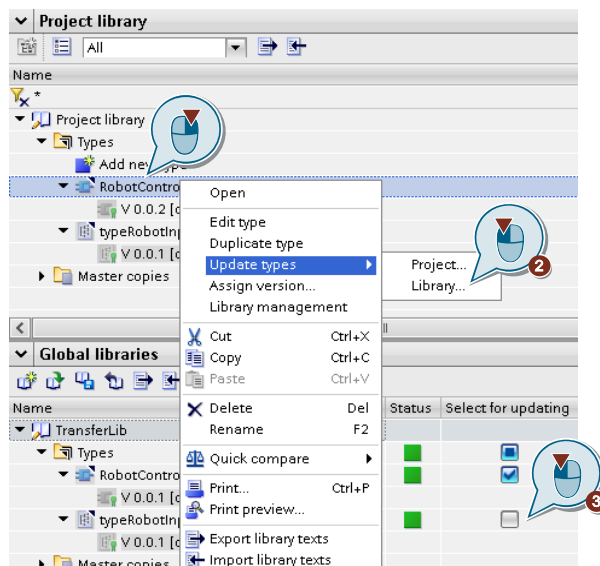


2. Choose the name, path, version, author and comment, then click "Create".



4.2.2 Copying typified elements into the global "transfer library"

1. Open your "developer project".
2. To take the elements you created or updated from the project library of your "developer project" and integrate them into the global "transfer library", right-click on the elements of the project library and select "Update types > Project".
3. Uncheck the box in the column for "Select for updating" for all types that were not modified. This way, the integrator will know which types to update.



Result

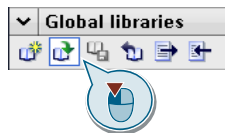
- The typified block (including subordinate elements) is located in the global "transfer library".
- Repeat the procedure until all newly created and modified elements from the project library of your "developer project" are copied into the "transfer library".
- You can save the global "transfer library" and deliver it to the integrator for integration into the "corporate library".

5 Applications in TIA Portal for "users"

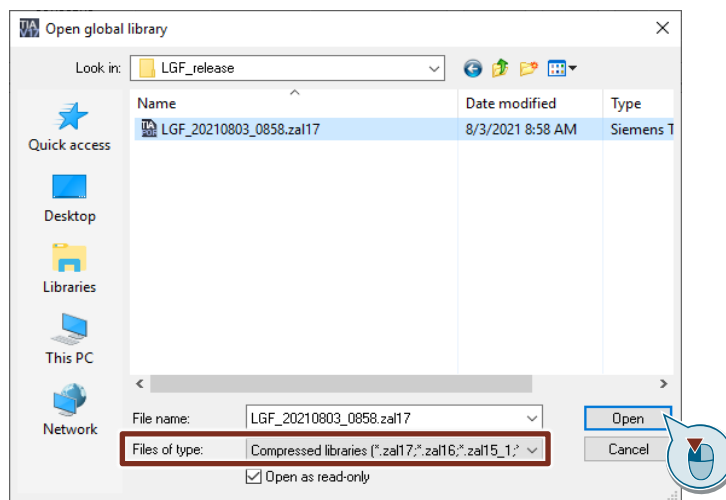
5.1 Working with the "plant project"

5.1.1 Opening the archived global "corporate library"

1. Open your "plant project".
2. Under "Global libraries", click "Open global library".



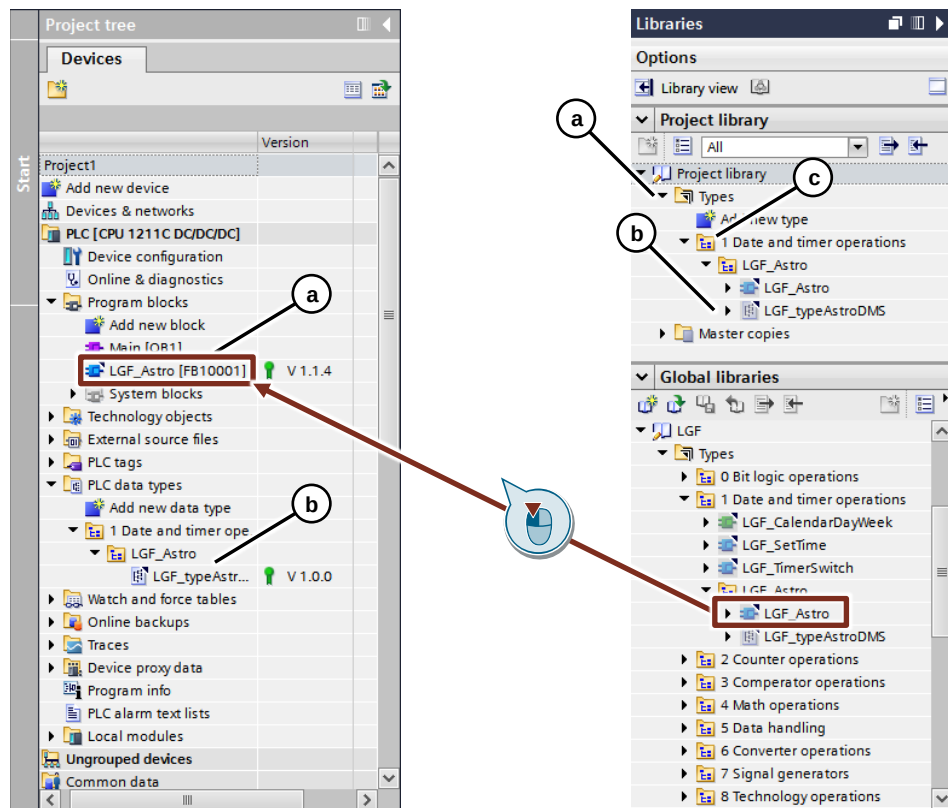
3. In the "Files of type" dropdown menu, select "Compressed libraries". Navigate to the archived "corporate library" and click "Open".



4. Select the storage path for the extracted "corporate library" and confirm the dialog with "OK". The extracted "corporate library" will open automatically.

5.1.2 Adding typified library elements to "plant project"

1. Open your "plant project" and the global "corporate library".
2. Drag the typified element (here, FB LGF_Astro) into the "Program blocks" folder.



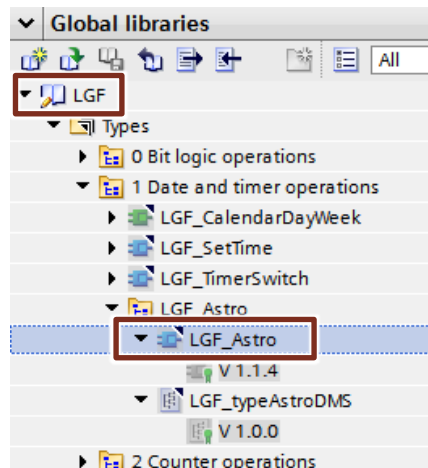
- a. The block that you added will be stored in the PLC program and in the "project library" under "Types".
- b. If a typified library element has a subordinate type, this will also be instantiated (here: "LGF_typeAstroDMS" is a subordinate type to "LGF_Astro").
- c. If a typified library element is located in a folder structure in the global "corporate library", it will be carried over to the project.

5.1.3 Updating selected types in the "plant project"

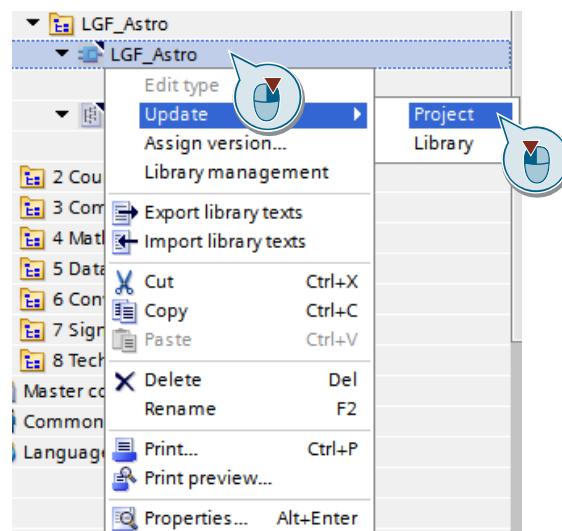
Requirements

Your "plant project" must contain typified blocks and the global "corporate library" must have a newer version of the typified blocks.

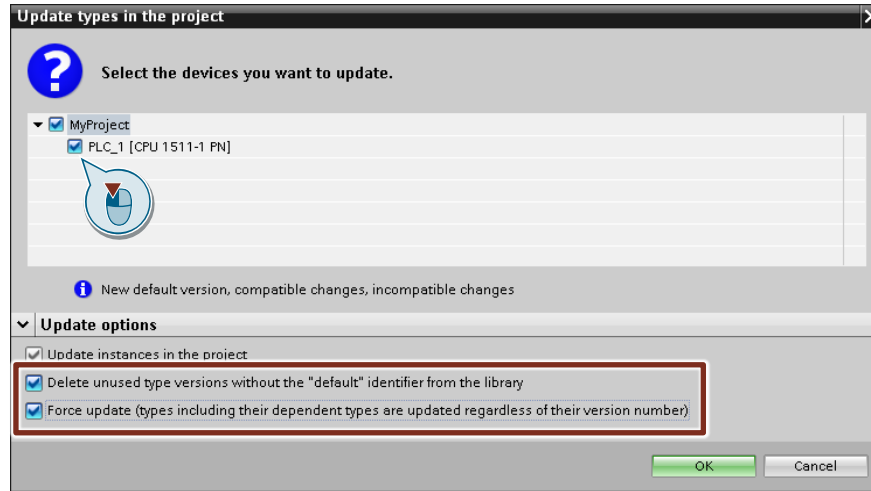
1. Open your "plant project" and the global "corporate library".
2. In the global "corporate library" (here: "LGF"), navigate to the typified element that you wish to update in your "plant project" (here: "LGF_Astro").



3. Right-click on the typified library element and select "Update types > Project".



4. In the dialog, select all PLCs in which the typified library elements should be updated. Select that you wish for unused (older) type versions to be deleted from the project library of your "plant project". Select the "Force update" option to make sure that the "plant project" matches the state of the "corporate library". Confirm the dialog with "OK".



Note For the purpose of consistency, always update all PLCs in the project.

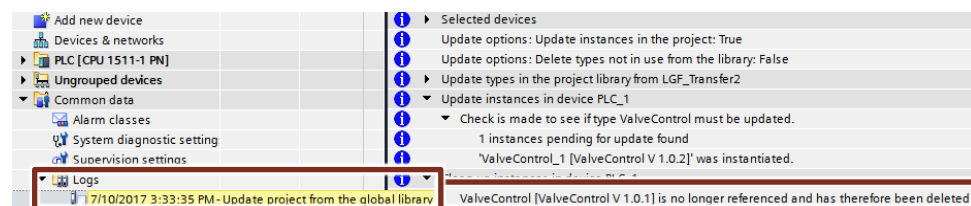
Result

All type instances of the selected library element have been updated in the selected PLCs and in the project library.

Note All typified library elements in the PLCs which are not called in the program or instantiated will be deleted with the update in the "plant project" program.

You have the option of tracing all processes performed during an update by going to "Common data > Logs".

Figure 5-1: Project logs in TIA Portal

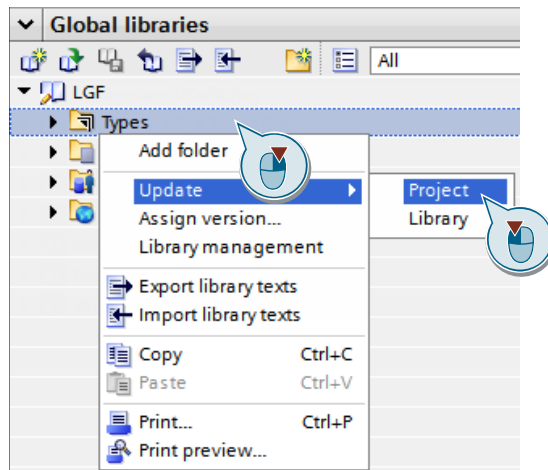


5.1.4 Updating all types in the "plant project"

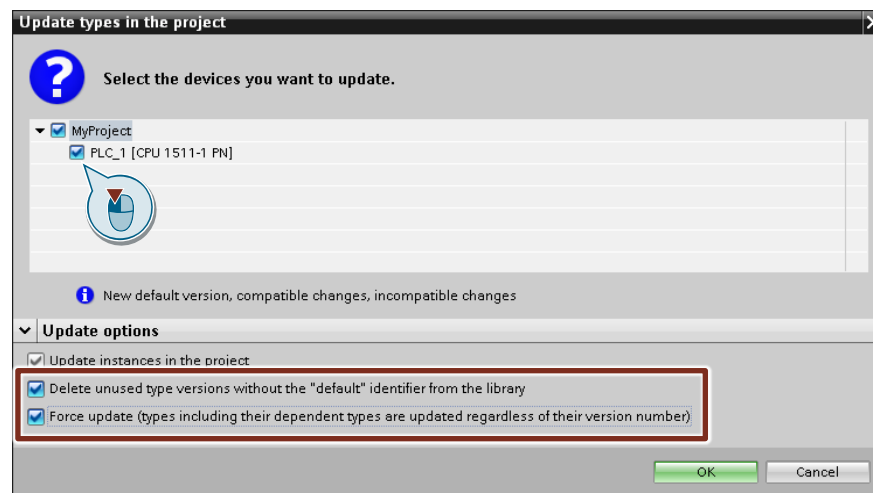
Requirements

Your "plant project" must contain typified blocks and the global "corporate library" must have a newer version of the typified blocks.

1. Open your "plant project" and the global "corporate library".
2. Right-click on the "Types" folder and select "Update types > Project".



3. In the dialog, select the PLCs in which the typified elements should be updated. Select that you wish for unused (older) type versions to be deleted from the project library of your "plant project". Select the "Force update" option to make sure that the "plant project" matches the state of the "corporate library". Confirm the dialog with "OK".



Note For the purpose of consistency, always update all PLCs in the project.

Result

All instances of all typified elements located in the global "corporate library" have been updated in the selected PLCs and in the project library of the "plant project". The status of all types is consistent, as can be seen from the green rectangle in the "Status" column.

CAUTION If the update causes an inconsistent state (error message, library "yellow"), stop the update and contact the person responsible for the library (integrator).

Note All typified library elements in the PLCs which are not called in the program or instantiated will be deleted with the update in the "plant project" program.

Note The update will copy all types from the "corporate library" to the project library.

Note If changes are made to the interface of library blocks, the **calls** of these library blocks may in certain cases (highlighted blue in [Table 5-1](#)) not be updatable by compiling or with "Update inconsistent block calls". Instead, every **call** will have to be updated manually with the context menu.

Table 5-1:

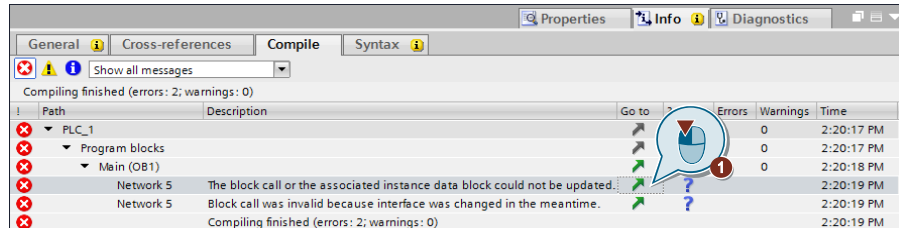
	Updating with context menu	Updating with "Update inconsistent block calls" button	Implicit update during compilation
Parameters added	X	X	X
Parameters removed Current parameter not interconnected	X	X	X
Parameters removed Current parameter interconnected	X	-	-
Parameters renamed Current parameter not interconnected	X	X	X
Parameters renamed Current parameter interconnected	X	-	-

Procedure in case of changes to the block interface of library blocks

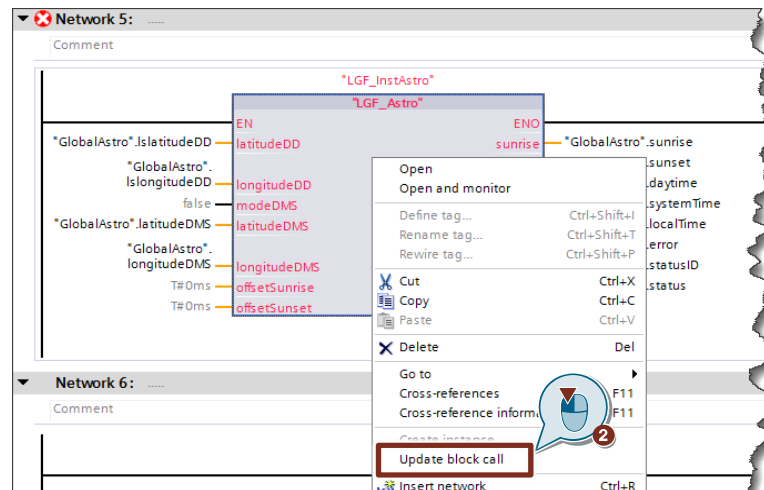
If the interface of library blocks has been changed, in some cases the plant project might compile with errors after the update. Every call will have to be updated manually with the context menu.

Follow the steps below:

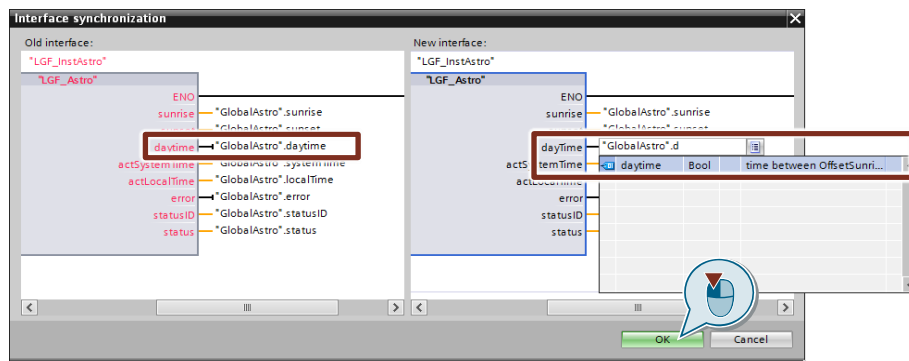
1. Compile your project. To open the call location of the library block, click the "Go to" button next to the error message.



2. Right-click on the block call and select "Update block call".



3. The following dialog shows the interface comparison between the old block and the new block. Check the interconnected formal parameters and correct them directly where necessary, e.g. with Intellisense (see Figure below). The tag can also be moved from the left side to the right side with copy & paste or drag & drop. Click "OK" to close the dialog.

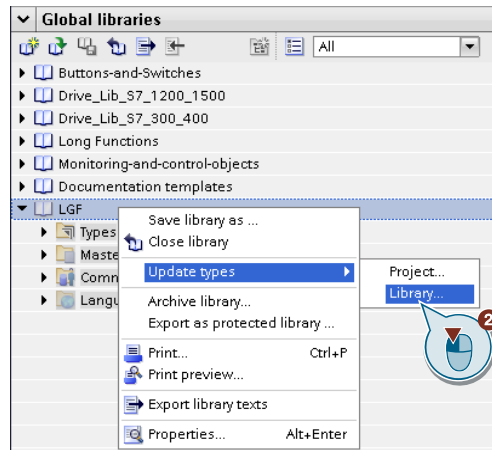


The block call will be updated.

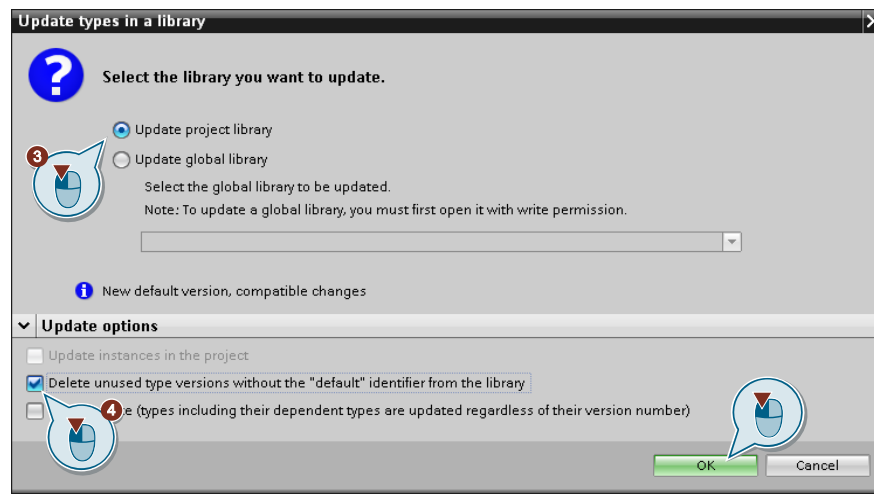
5.1.5 Replacing discontinued types

Proceed as follows to replace discontinued typed blocks:

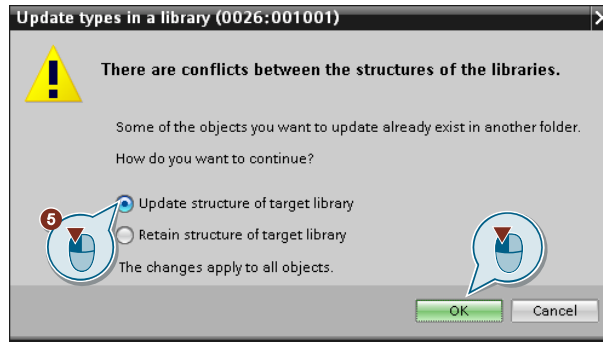
1. Open your "plant project" and the global "corporate library".
2. Right-click on the "Types" folder in the global library and select "Update types > Library...".



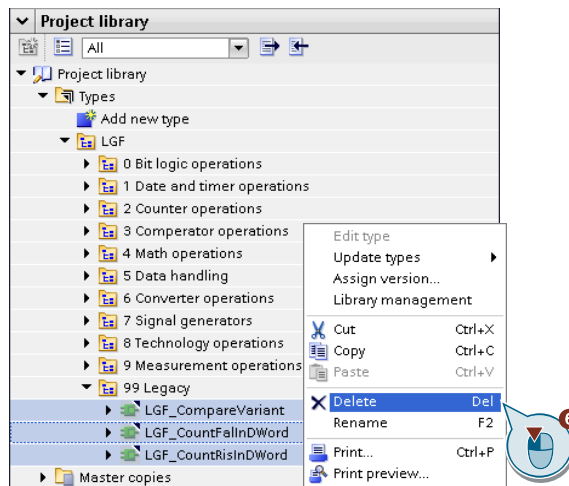
3. Select "Update project library" in the dialog.
4. Enable the option "Delete unused type versions without the "default" identifier from the library" in order to delete unused type versions from the project library of your "plant project". Confirm the dialog with "OK".



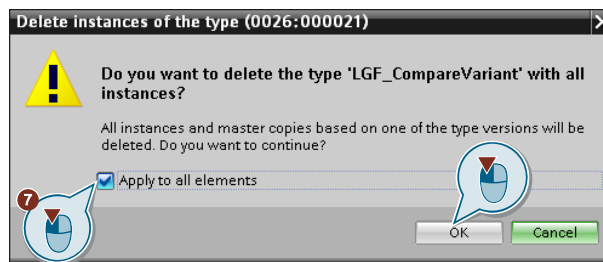
5. Select "Update structure of target library" to conform the structure of the project library to the structure of the global "corporate library". Confirm the dialog with "OK".



6. Delete the obsolete blocks in the "Legacy" folder. Confirm deletion in the next dialog.



7. In the next dialog, select "Apply to all elements". Confirm the dialog with "OK".



Result

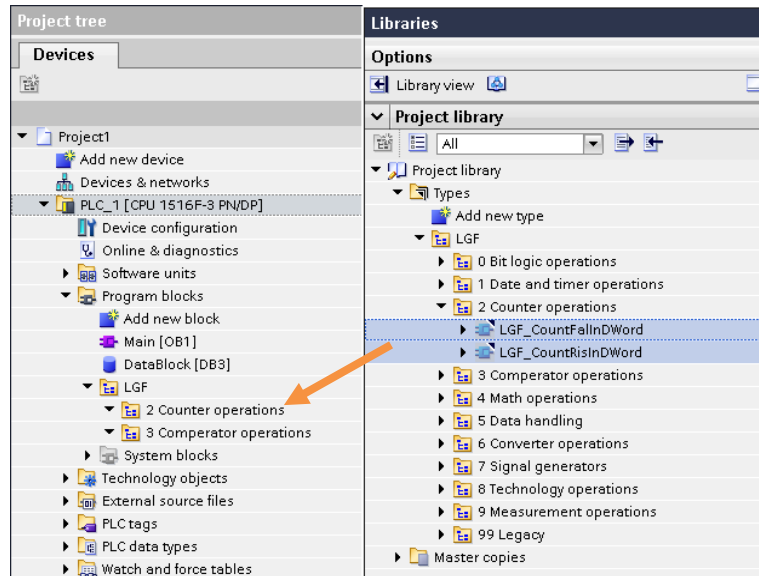
The new blocks are transferred from the global "corporate library" to the project library.

The obsolete typed blocks and all instances are deleted from the project library and the "plant project".

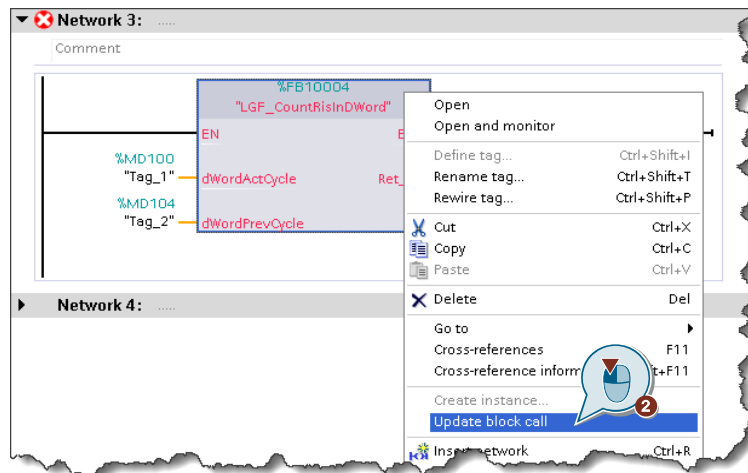
Example 1: Replacing obsolete blocks in a project with a new type

In the example below, the outdated FC "LGF_CountRisInDWord" is replaced by an FB of the same name:

1. Drag and drop the new types from the project library into the "plant project".

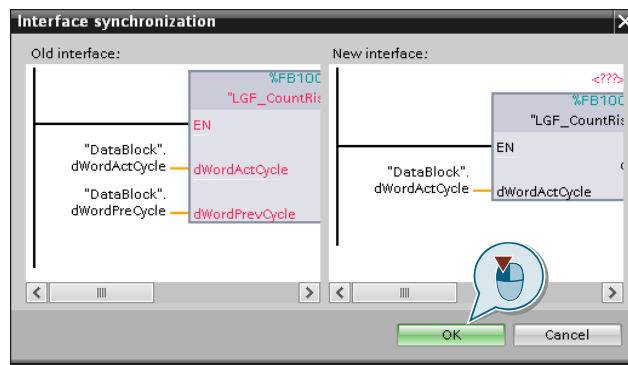


2. Open the instances in the program, right-click on the block call and select "Update block call".

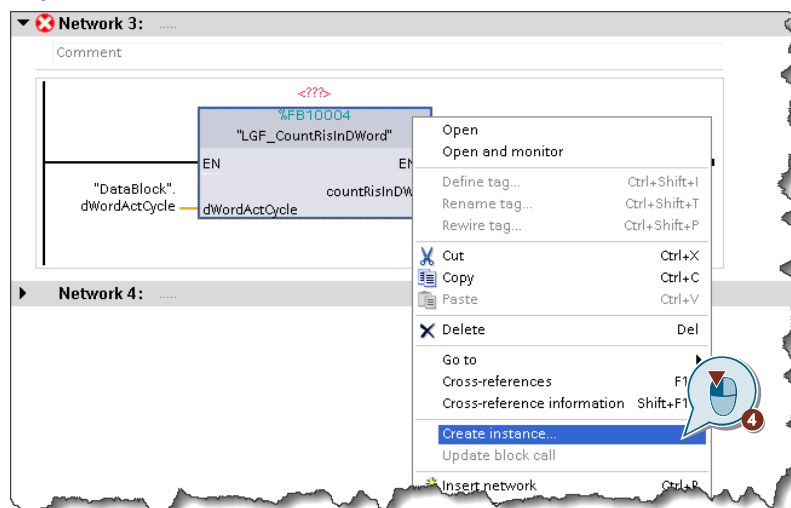


- The following dialog shows the interface comparison between the old block and the new block. Check the interconnected formal parameters and correct them directly where necessary, e.g. by drag and drop.

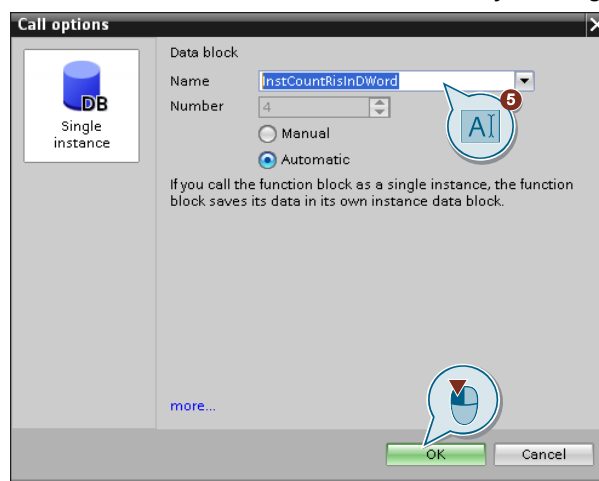
Click "OK" to close the dialog.



- Right-click on the new block call and select "Create instance".



- Give a new instance name and confirm by clicking "OK".



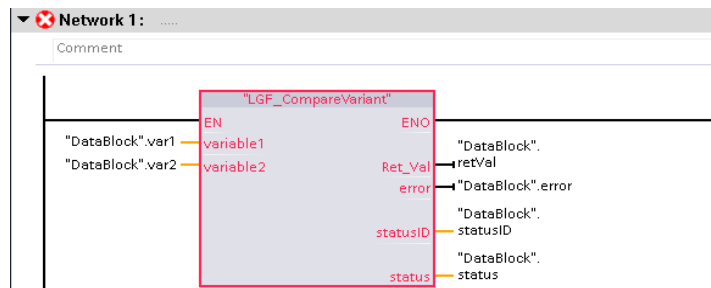
Result

The block call of the old block is replaced by the block call of the new block.

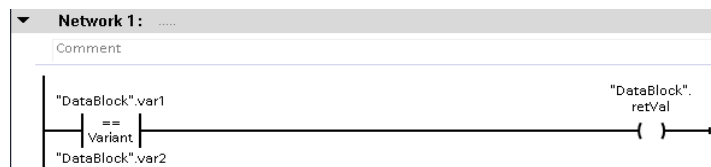
Example 2: Replacing obsolete blocks in a project with new system functions

In the following example, the FC "LGF_CompareVariant" is replaced by a comparator and an assignment.

1. Open the points of use in the program.



2. Replace the block with a comparator and an assignment.



In the SCL programming language you will use the IF instruction for this.

```

1 IF ("DataBlock".var1 = "DataBlock".var2)
2 THEN
3   "DataBlock".retVal := TRUE;
4 ELSE
5   "DataBlock".retVal := FALSE;
6 END_IF;

```

6 Creating customized libraries

If you wish to create a custom global library (company- or department-specific) from multiple basic libraries, you must adhere to the following sequence so as to avoid inconsistencies and keep a manageable structure. The workflow is similar to the workflow for creating "corporate libraries".

Example for a basic library:

Library with general functions (LGF) for STEP 7 (TIA Portal) and S7-1200 / S7-1500

<https://support.industry.siemens.com/cs/ww/en/view/109479728>

The following terms are adapted for the workflow.

- Basic library
→ Corresponds to a global "corporate library"

The following terms are appended:

- "company-specific development library"

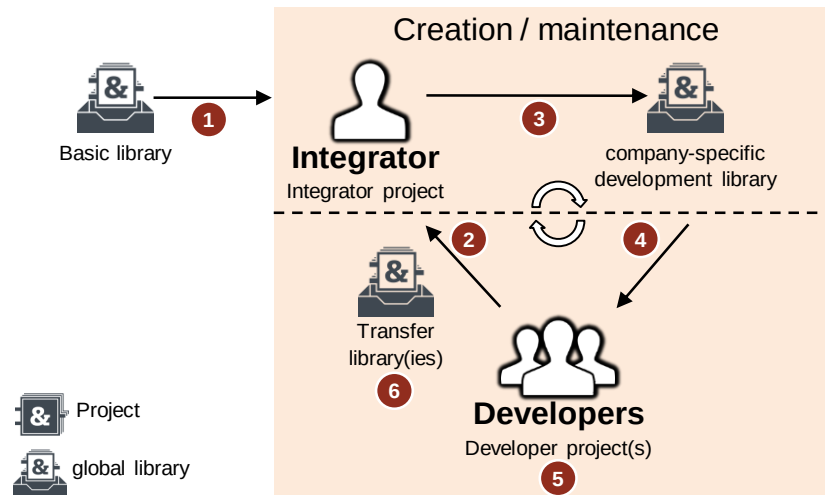
All other original terms (e.g. roles) remain unchanged.

Recommendation

- Use folder structures in your "company-specific development library" for a better overview.
- Create a folder in your "company-specific development library" for each basic library.

Workflow for integrating a basic library into a company-specific library

Figure 6-1: Workflow



1. The "integrator" integrates all or only selected library elements of the "basic library" into the "integrator project". The integrator structures all elements within an appropriate folder structure.
→ Chapter [3.1 Working with the "integrator project"](#)
2. The "integrator" integrates all the delivered library elements of the "transfer libraries" into the "integrator project".
→ like in Step 1
3. The "integrator" creates the next version of the "company-specific development library" from the "integrator project".
→ Chapter [3.2 Working with the global "development library"](#)
4. The "integrator" provides the current version of the "company-specific development library" to the "developers" as a global library.
5. The "developers" update their "test projects" with the current "company-specific development library". They create or update library elements. The library elements receive a new version.
→ Chapter [4.1 Working with the "developer project"](#)
6. The "developers" deliver created or updated library elements to the "integrator" in "transfer libraries".
→ Chapter [4.2 Working with the global "transfer library"](#)

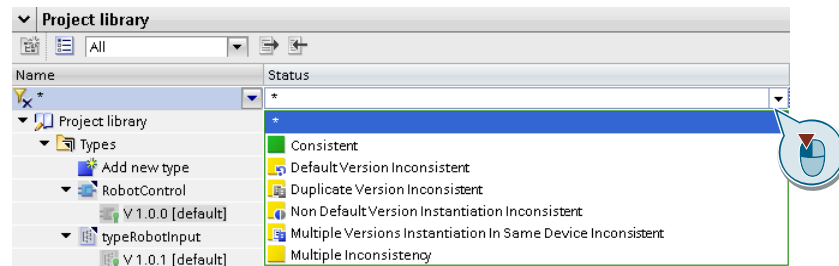
The development cycle can be repeated as often as necessary.

7 Troubleshooting

7.1 Inconsistent library status

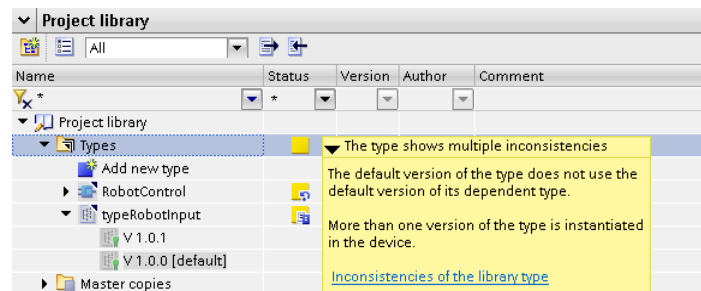
The status gives you a quick overview of the correct use of the library elements.

Figure 7-1: Library status



The library status is propagated through the folder levels. Use the tooltip function or mouse over to obtain additional information.

Figure 7-2: "Multiple Inconsistencies"



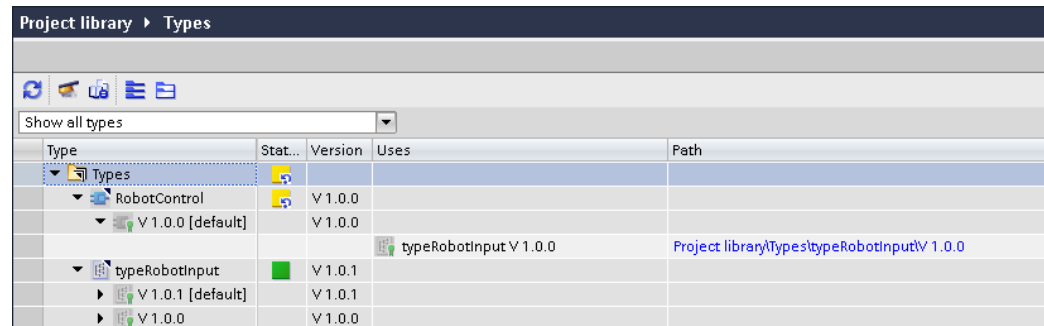
Fixing of various inconsistencies is described in the following chapters.

7.1.1 Inconsistency in the default versions

When you work with nested library types, changes to one type can affect dependent types. The library status will inform you of inconsistencies.

This function is demonstrated on the basis of the example below. The FB "RobotControl" is not yet using the modified default version of the PLC data type "typeRobotInput".

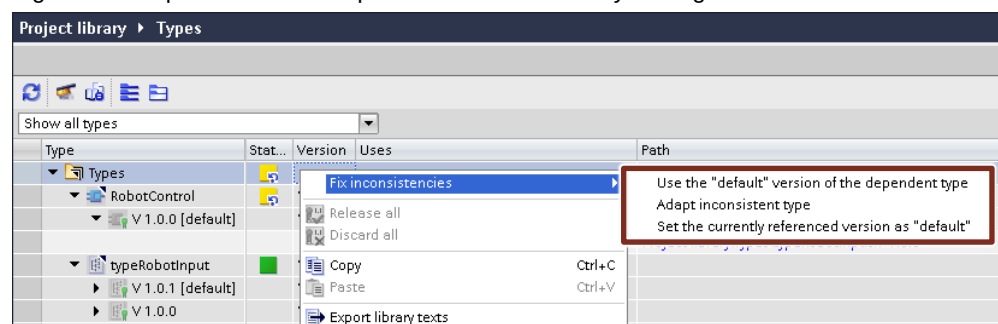
Figure 7-3: Representation of dependencies in the Library management



Type	Stat...	Version	Uses	Path
Types				
RobotControl		V 1.0.0		
V 1.0.0 [default]		V 1.0.0		
typeRobotInput		V 1.0.1	typeRobotInput V 1.0.0	Project library/Types/typeRobotInputV 1.0.0
V 1.0.1 [default]		V 1.0.1		
V 1.0.0		V 1.0.0		

When the default version of the type in use does not use the default version of its dependent type, then you can remedy the inconsistencies with the following functions.

Figure 7-4: Representation of dependencies in the Library management



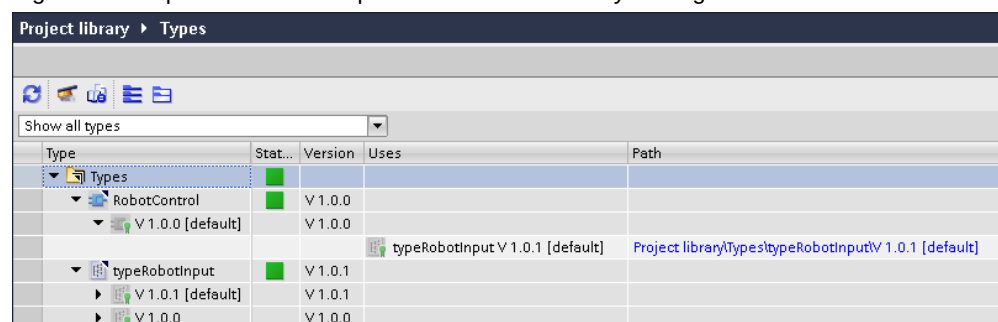
Type	Stat...	Version	Uses	Path
Types				
RobotControl		V 1.0.0		
V 1.0.0 [default]		V 1.0.0		
typeRobotInput		V 1.0.1		
V 1.0.1 [default]		V 1.0.1		
V 1.0.0		V 1.0.0		

Use default version

Use this function in the event of compatible changes, e.g. comment or block logic.

In the "Fix inconsistencies" context menu, select the function "Use the "default" version of the dependent type".

Figure 7-5: Representation of dependencies in the Library management



Type	Stat...	Version	Uses	Path
Types				
RobotControl		V 1.0.0		
V 1.0.0 [default]		V 1.0.0		
typeRobotInput		V 1.0.1	typeRobotInput V 1.0.1 [default]	Project library/Types/typeRobotInputV 1.0.1 [default]
V 1.0.1 [default]		V 1.0.1		
V 1.0.0		V 1.0.0		

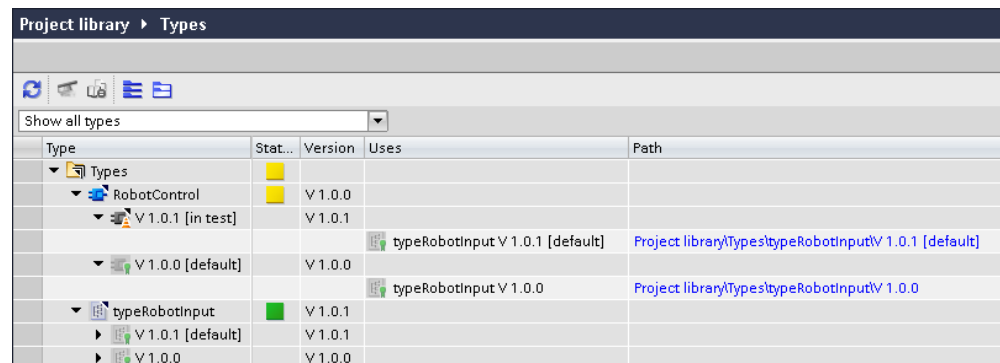
The library is now consistent. The FB "RobotControl" is now using the new default version of the PLC data type "typeRobotInput".

Adapting an inconsistent type

Use this function in the event of incompatible changes, such as changes to the interface.

In the "Fix inconsistencies" context menu, select the function "Adapt inconsistent type". The selected type is now in [in test] and now references the new default version of the PLC data type. You now have the ability to adapt the type accordingly.

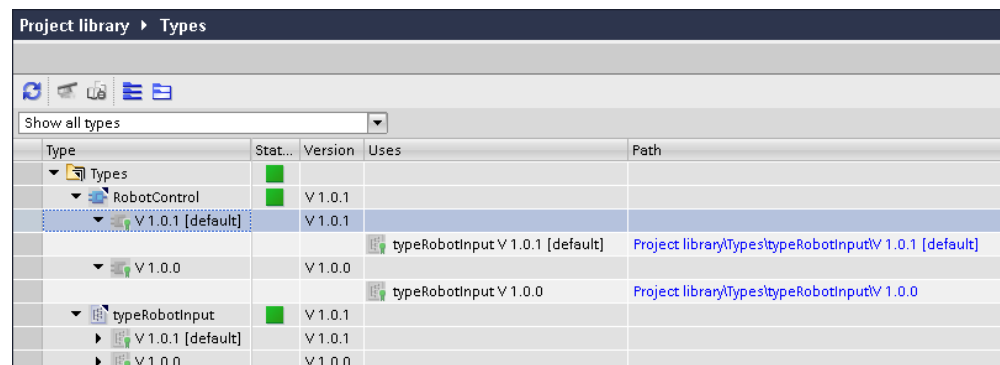
Figure 7-6: Representation of dependencies in the Library management



Type	Stat...	Version	Uses	Path
Types				
RobotControl		V 1.0.0		
V 1.0.1 [in test]		V 1.0.1	typeRobotInput V 1.0.1 [default]	Project library\Types\typeRobotInput\V 1.0.1 [default]
V 1.0.0 [default]		V 1.0.0	typeRobotInput V 1.0.0	Project library\Types\typeRobotInput\V 1.0.0
typeRobotInput		V 1.0.1		
V 1.0.1 [default]		V 1.0.1		
V 1.0.0		V 1.0.0		

Release the FB "RobotControl".

Figure 7-7: Representation of dependencies in the Library management



Type	Stat...	Version	Uses	Path
Types				
RobotControl		V 1.0.1		
V 1.0.1 [default]		V 1.0.1	typeRobotInput V 1.0.1 [default]	Project library\Types\typeRobotInput\V 1.0.1 [default]
V 1.0.0		V 1.0.0	typeRobotInput V 1.0.0	Project library\Types\typeRobotInput\V 1.0.0
typeRobotInput		V 1.0.1		
V 1.0.1 [default]		V 1.0.1		
V 1.0.0		V 1.0.0		

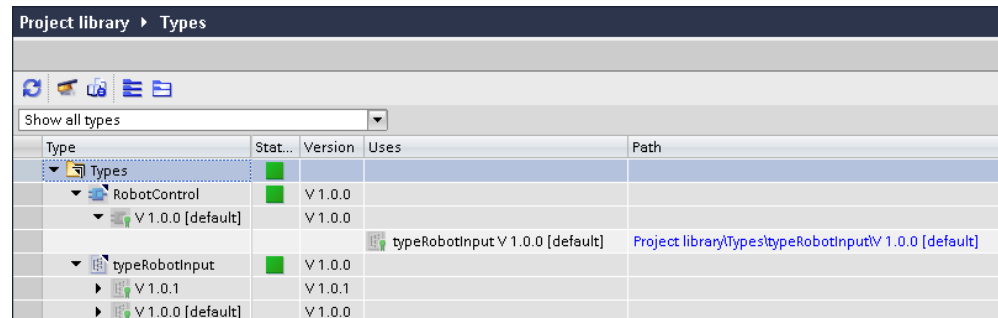
The library is now consistent. The adapted version of the FB "RobotControl" is using the new default version of the PLC data type "typeRobotInput".

Setting the currently referenced version as "Default"

Use this function to set the currently referenced version of the PLC data type as Default.

In the "Fix inconsistencies" context menu, select the function "Set the currently referenced version as "default"".

Figure 7-8: Representation of dependencies in the Library management



The screenshot shows the 'Project library > Types' window. It features a tree view on the left and a table on the right. The tree view shows a hierarchy: 'Types' (expanded) contains 'RobotControl' (expanded) which contains 'V 1.0.0 [default]'. Below this, 'typeRobotInput' (expanded) contains 'V 1.0.1' and 'V 1.0.0 [default]'. The table on the right has columns: 'Type', 'Stat...', 'Version', 'Uses', and 'Path'. It lists the following entries:

Type	Stat...	Version	Uses	Path
Types	■			
RobotControl	■	V 1.0.0		
V 1.0.0 [default]	■	V 1.0.0		
typeRobotInput	■	V 1.0.0	typeRobotInput V 1.0.0 [default]	Project library\Types\typeRobotInput\V 1.0.0 [default]
V 1.0.1	■	V 1.0.1		
V 1.0.0 [default]	■	V 1.0.0		

The library is now consistent. The FB "RobotControl" uses the original default version of the PLC data type "typeRobotInput".

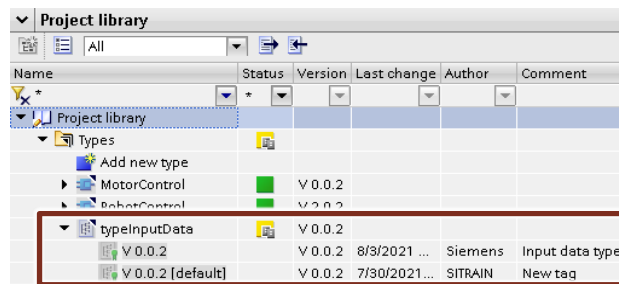
Note

You can also apply the "Fix inconsistencies" function to multiple types as a batch operation.

7.1.2 Inconsistency with same versions ("duplicate version")

The following situation results in a version conflict between typified elements:

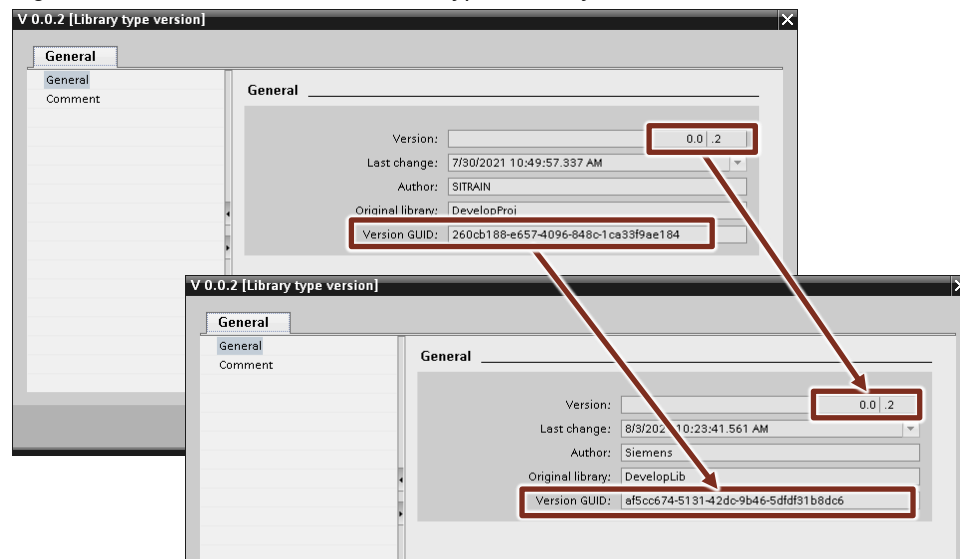
- You open a global library and copy a type (V1.0.0) to your project.
- You edit the type and create a new version (V1.0.1).
- You open another global library where the type has the same version (V1.0.1).
- When you attempt to integrate that type into your project, the project library displays the status "Duplicate version" for the type.



Name	Status	Version	Last change	Author	Comment
Project library					
Types					
Add new type					
MotorControl		V 0.0.2			
RobotControl		V 0.0.2			
typeInputData		V 0.0.2			
V 0.0.2		V 0.0.2	8/3/2021 ...	Siemens	Input data type
V 0.0.2 [default]		V 0.0.2	7/30/2021...	SITRAIN	New tag

If you compare the properties of the library element in the project with the library element in the global library, you will see that they have the same "Version" but not the same "Version GUID". Therefore, the two library elements are different.

Figure 7-9: Version conflict between two typified library elements



Solution (integrator)

- The "integrator" increments the version of the library element in the project library of the integrator's "integrator project" and then in the global "development library" for the "developers".

Solution (developer)

- The "developer" updates the library element in his/her "developer project" with the global "development library" that was updated by the "integrator" (see chapter [4.1.1 Updating the "developer project" with the global "development library"](#)).

Solution (user)

- The "user" is not permitted to make changes to types.
- The "user" compares the various types in the project library of the "plant project" in order to ascertain functional changes. If there are differences, it is recommended to contact the publisher ("integrator") of the "corporate library".
- The "user" updates the "plant project" with the "Force update" option and "Delete unused type versions without the "default" identifier from the library".

7.1.3 "Non Default Version" inconsistency

From the status of the library block in the project library, as well as from the project tree, it is possible to see that the project is not using the "Default" version.

Figure 7-10: "Non Default Version" inconsistency

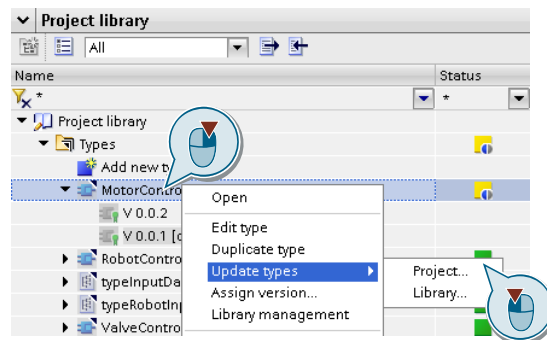


Note

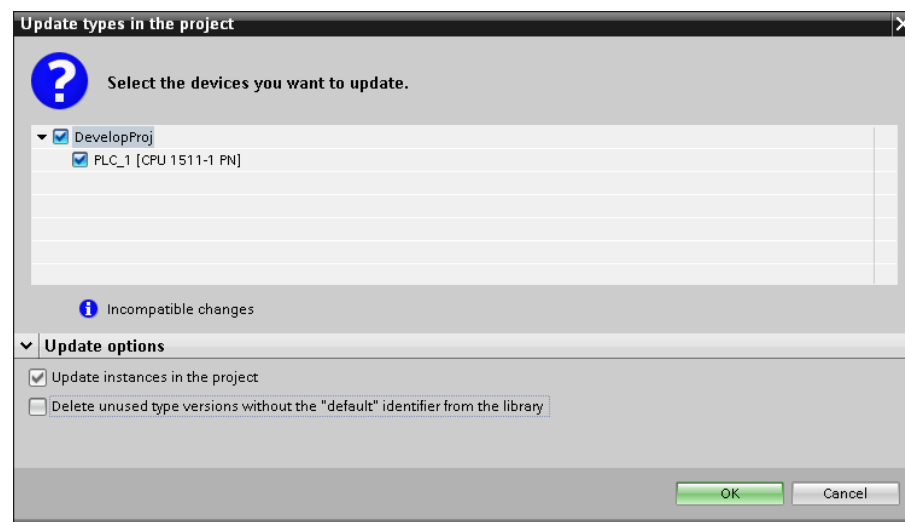
This library status is also displayed for library elements that are [in test]. To fix the inconsistency, release the type or discard the changes.

To fix the "Non Default Version" inconsistency, proceed as follows:

1. Update the project. To do this, right-click on the type and select "Update types > Project".



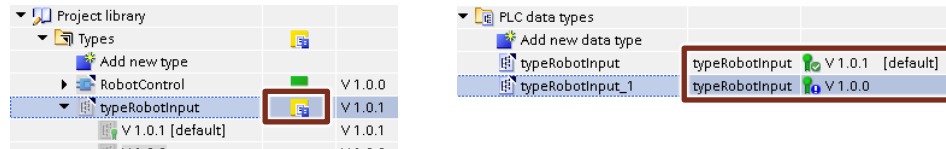
2. In the dialog, select all PLCs where the modified version must be updated.



7.1.4 "Multiple Versions Instantiation" inconsistency

From the status of the library block in the project library, and in the project tree, it is possible to see that multiple versions of the same type are being used in one device.

Figure 7-11: "Multiple Versions Instantiation" inconsistency



To fix the inconsistency, perform a library update from the project library. See chapter [7.1.3](#).

7.2 Resolving multiple versions

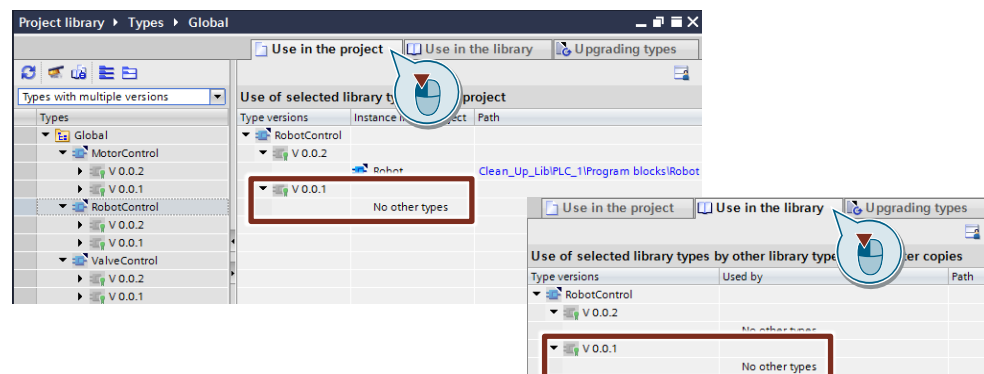
If the project library or the global libraries contain multiple versions of the same type, reduce the types to the current version. Use the Library management in the Library view to clean up the library. Before cleaning up, check whether older versions are used in the project, in the project library or in the global library.

Further information can be found in chapter [8.5 Library view and Library management](#).

Preparation

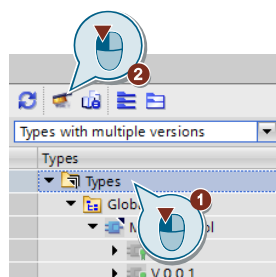
1. Open the Library view.
2. Enable Library management, focusing on the project library.
3. Enable the filter "Types with multiple Versions".

Case 1: Older type not used in project or project library

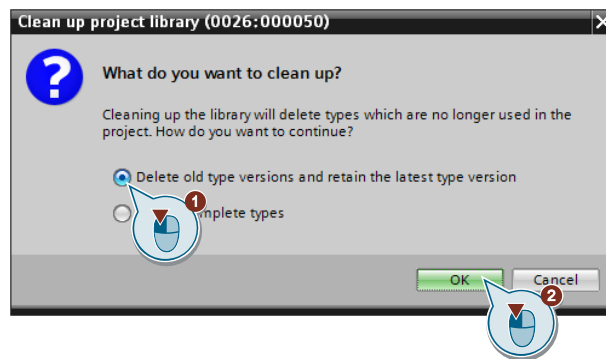


Solution (integrator, developer, user)

1. In the project library, select the "Types" folder and click "Clean up library".

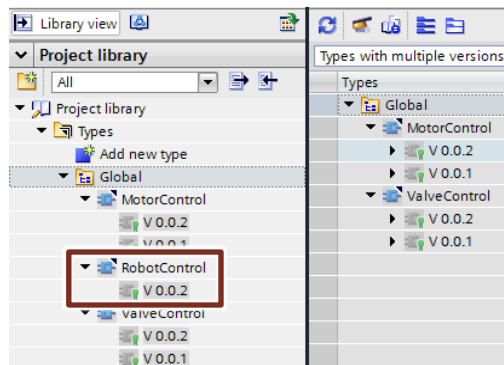


- In the dialog, select "Delete old type versions and retain the latest type".



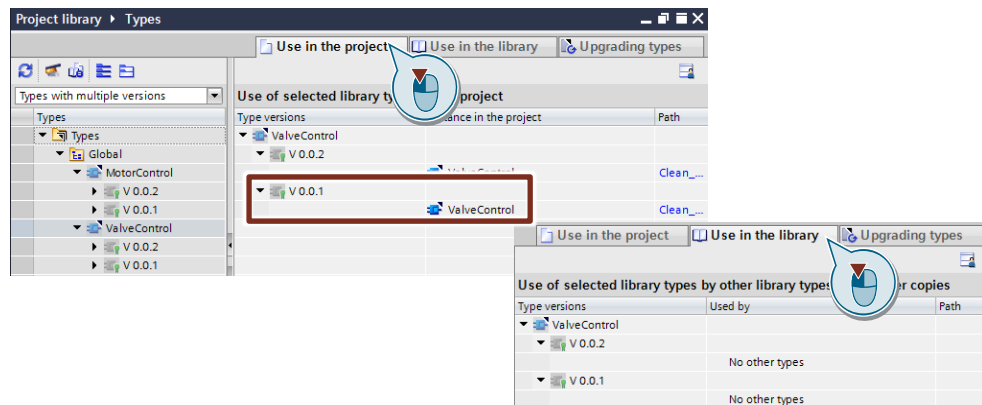
Result

The type "RobotControl" now only has one version in the project library and will no longer appear in the Library management as a type with multiple versions.



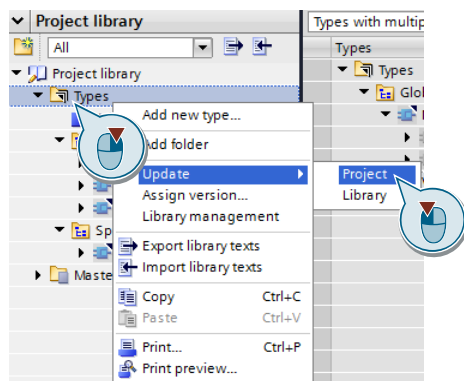
The type types "MotorControl" and "ValveControl" still exist in multiple versions. The cause for this can be found in the following cases.

Case 2: Older type still used in project

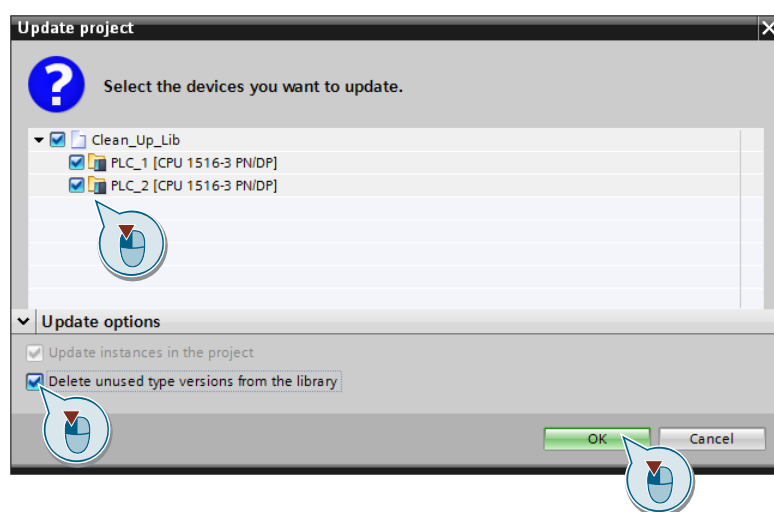


Solution (integrator, developer, user)

1. In the project library, right-click on the "Types" folder and select "Update types > Project".

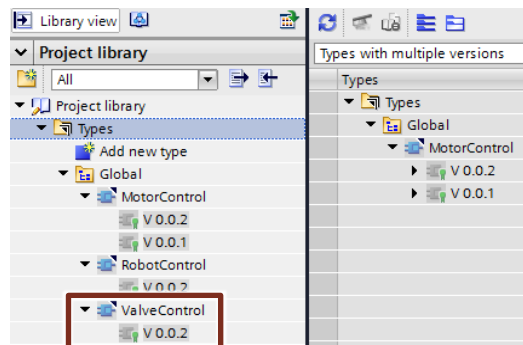


2. Select all PLCs that you wish to update, enable the option "Delete unused type versions from the library" and confirm the dialog with "OK".



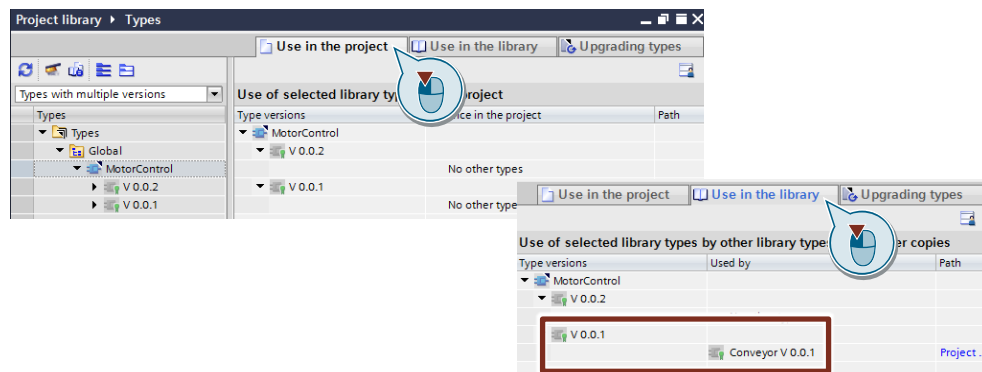
Result

The type "ValveControl" now only has one version in the project library and will no longer appear in the Library management as a type with multiple versions.



The type "MotorControl" still exists in multiple versions. The reason for this lies in the case below.

Case 3: Old version used in superordinate library type



Solution (integrator)

Note

In this case, the higher-level type must be updated so that it uses the new version.

Consult with the "developer" working on the type to find a solution to this conflict.

1. Instantiate the superordinate block (here, "Conveyor") in your project, if it does not exist.
2. Follow the step-by-step instructions in chapter [3.1.2 Updating existing library elements from the "transfer libraries" in the "integrator project"](#).
3. Follow the step-by-step instructions in chapter [3.2.3 Deleting older versions of types in the global "development library"](#).

7.3 Undesired assignment of a new version

If you have assigned a new version to a type in your project but didn't want to do this, you can restore your project to the original state as follows.

Case 1: Old version is present in the project library

Solution (integrator, developer)

1. Set old version as default version.
2. Fix any inconsistencies (if applicable) if the type is used in superordinate types.
3. Update project.

Result

The original type will be used in the project.

Case 2: Old version is present in the global library

Solution (integrator, developer)

1. In the global library, navigate to the original type and update the project with the option "Force update" and "Delete unused type versions without the "default" identifier from the library".
2. Fix any inconsistencies (if applicable) if the type is used in superordinate types and contains an incompatible change.
3. Update project.

Result

The original type will be used in the project.

7.4 Undesired disconnection of an instance from its type

If you have disconnected a type instance from its associated type, you cannot undo this action because re-typifying will lead to a completely new type.

Solution (integrator, developer, user)

1. Delete the element in the project that was originally a type instance.
2. In the project library, navigate to the original type and instantiate it in your project.

Result

The element has been reinstantiated in your project as a type.

7.5 Undesired deletion of a type instance

You have deleted a type instance, but you didn't mean to do this.

Solution (integrator, developer, user)

1. In your project library or global library, navigate to the type pertaining to the type instance that was mistakenly deleted.
2. Instantiate the type in your project.

Result

The accidentally deleted type instance is in your project again.

7.6 Deleted types in global libraries are not deleted from project

You receive a new version of a global library. The new version no longer contains various blocks; you wish to delete these block from your project as well.

Solution (integrator, developer, user)

1. Read the change history to identify the deleted types.
2. Manually delete the types in the project library. If the types are instantiated in the project, delete these manually as well.
3. Replace the deleted types as described in the change history.

Result

Your project only uses the types that are in the current "corporate library".

8 Useful information / fundamentals

8.1 Terms

Project library

Every TIA Portal project has its own library, known as the project library. In the project library, you can store objects you want to use more than once in the project. The project library is opened, saved, and closed together with the current project.

Global libraries

Global libraries are independent of any specific project and can be distributed to other users.

Common access to global libraries, for example on a network drive, is also possible if all users open the global library in write-protected mode. TIA Portal automatically manages the global library. Once a newer version of an existing global library is available, you will receive a prompt to update the corresponding global library to the newer version.

Global libraries that have been created in earlier versions of TIA Portal can continue to be used. To do this, you must upgrade them.

Note

Handling of global libraries is described in the following articles:

How do you open libraries in STEP 7 (TIA Portal)?

<https://support.industry.siemens.com/cs/ww/en/view/37364723>

How can you open a global library with write permissions in STEP 7 (TIA Portal)?

<https://support.industry.siemens.com/cs/ww/en/view/37364723>

When starting TIA Portal V13 and higher, how do you get a global library to open automatically and use it as corporate library, for example?

<https://support.industry.siemens.com/cs/ww/en/view/100451450>

How do you update corporate libraries in STEP 7 (TIA Portal)?

<https://support.industry.siemens.com/cs/ww/en/view/109739199>

Master copies

Master copies are elements in the library which are not typified. They have no connection to the instances in the project where they are used.

Types

Types are versionable library elements; they support a professional development cycle. Projects in which the types are used can be updated as soon as new versions of the types are available.

Type instances

The instances where typified library elements are used in a project are known as type instances.

8.2 Overview of all library elements

Not every library element can be specified as a type or master copy.

Table 8-1: Library elements

Library element	Type	Master copies
SIMATIC PLC		
OB	-	X
FB	X	X
FC	X	X
DB (Global)	-	X
Technology objects	-	X
PLC tags	-	X
PLC data types	X	X
Watch and force tables	-	X
Traces	-	X
Text lists	-	X
SIMATIC WinCC Advanced / Professional		
Screens	X	X
Faceplates	X	-
Templates (of screens)	-	X
Pop-up screens	-	X
Slide-in screens	-	-
HMI tags	-	X
Scripts	X	X
Logs	-	X
HMI UDT	X	-
HMI style	X	-
HMI style sheet	X	-
User administration: User	-	X
User administration: Groups	-	X

8.3 Differences in types between PLC and HMI

Depending on the system, there are differences between the typable elements with the PLC and the HMI:

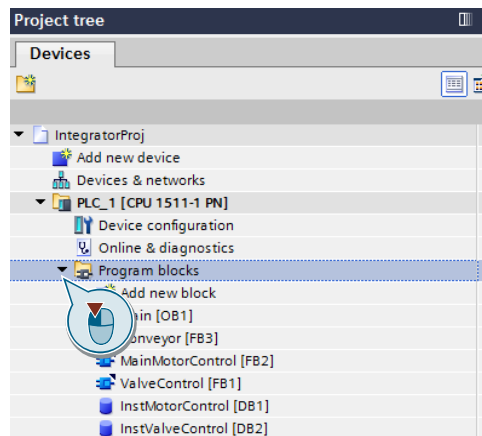
Table 8-2: Type differences with controller and HMI

PLC	HMI
Subordinate control elements are typed when the superordinate element is typed.	Subordinate HMI elements are not co-typified.
Subordinate control elements are instantiated when the superordinate element is instantiated.	Subordinate HMI elements are not co-instantiated.
Control elements are edited in the program of a PLC. A device is always required.	HMI screens and HMI scripts are edited in an HMI. Faceplates and HMI UDTs are edited directly in the library.

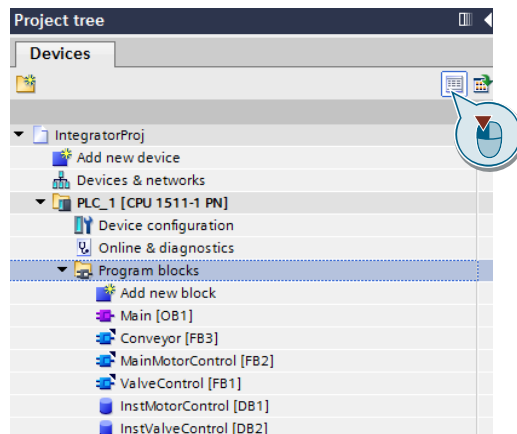
8.4 Displaying type information in the project tree

Proceed as follows to display the type information (name, version) in the project tree.

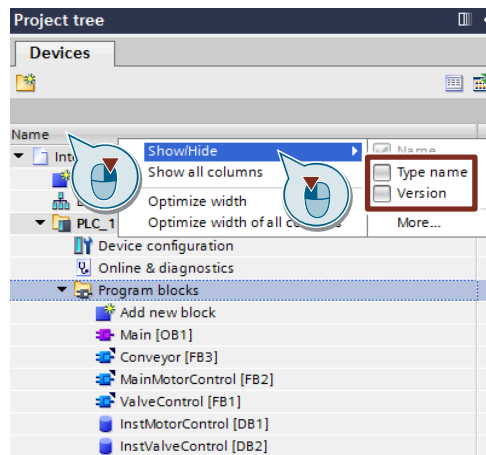
1. Open TIA Portal and your project.
2. Expand the "Program blocks" folder.



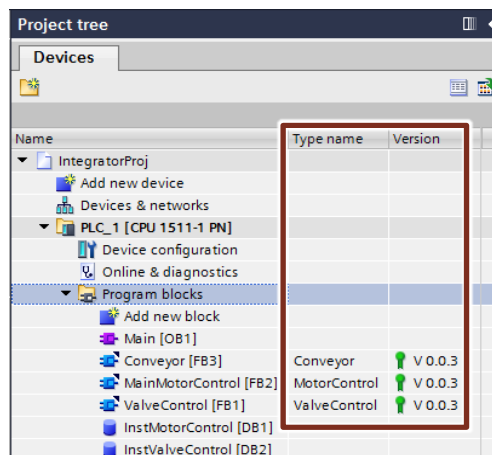
3. Click the "Display/hide column header" button.



4. Right-click on the column header, select "Show/Hide" and tick the boxes for "Type name" and "Version".



5. The type information will appear in the project tree for all type instances.

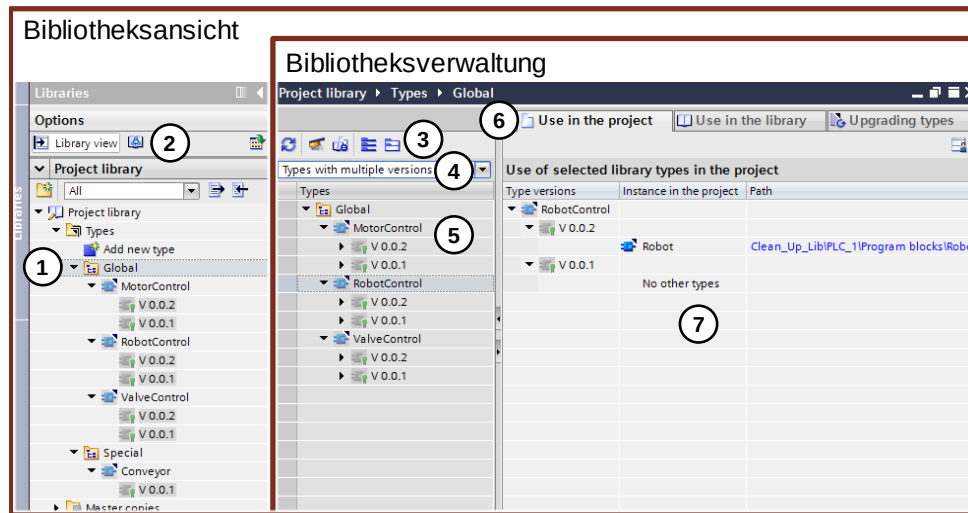


8.5 Library view and Library management

8.5.1 Overview

The Library view shows you the elements of a library in various views. For example, in the detail view you can see additional properties of the individual elements.

Figure 8-1: Library view and Library management



1. Library navigation
2. Buttons
 - "Library view open/close"
 - "Library management" button
The Library management will open differently depending on where you have set the focus in the Library navigation.
3. Buttons
 - "Update uses"
 - "Clean up library"
 - "Harmonize project"
 - "Expand all entries"
 - "Collapse all entries"
4. Filter dialog (see chapter [8.5.2 Filter dialog in the Library management](#))
5. Types matching the filter
6. Tab
 - "Use in the project"
 - "Use in the library"
 - "Upgrading types"
7. Instances of the types matching the filter

8.5.2 Filter dialog in the Library management

The filter dialog consists of multiple filters for displaying the types with various statuses and mutual dependencies.

- "Show all Types"
This filter shows all types.
- "Types with pending changes"
This filter shows all types that are in the [in test] status.
If you wish to finalize a project or a global library, use this filter to check whether all types are released.
- "Released Types"
This filter shows all published types.
- "Types with multiple versions"
This filter shows all types that have multiple versions.
Recommendation: Delete old versions of the types to improve your overview of the library and avoid inconsistencies.
- "Types not used in project"
This filter shows all types that have no instance in the project.
Recommendation: Check whether all types in the project are instantiated.
- "Types with inconsistencies in the "default" version"
This filter shows all types whose default versions do not reference the default versions of the dependent types.
Recommendation: Check whether superordinate types can be updated and make sure the library is consistent. See chapter: [3.1.2 Updating existing library elements from the "transfer libraries" in the "integrator project"](#).
- "Highest version of the type without "default" identifier"
This filter shows the highest version of any type that does not have the identifier "default".

8.5.3 Showing and hiding columns in the "Libraries" task card

If needed, you can show or hide additional columns in the project library or in global libraries. The additional columns can show the version number of a type belonging to an instance, for example.

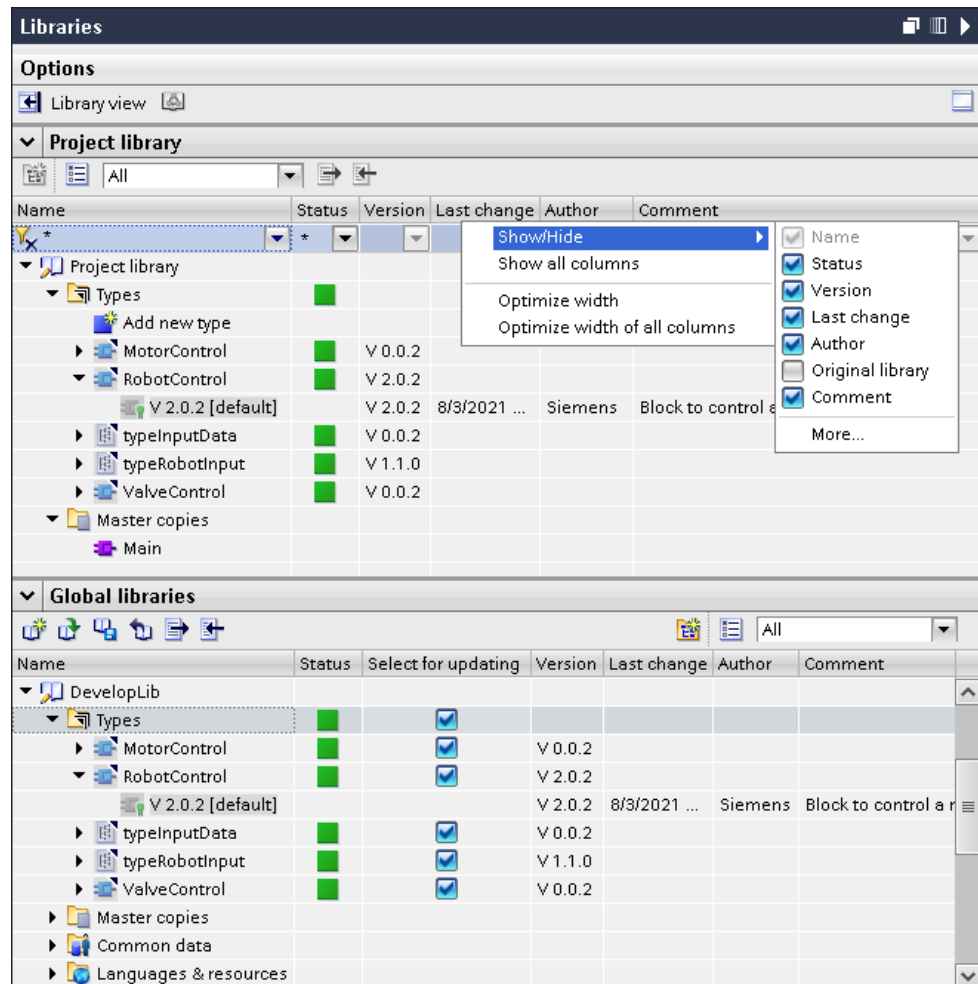
Proceed as follows to show or hide columns in the project library or in global libraries:

1. Right-click on the table header in the library.
2. In the context menu, select the command "Show/Hide" and select the columns that you wish to display.

Alternatively, select the columns that you wish to hide.

The desired columns will be shown or hidden.

Figure 8-2



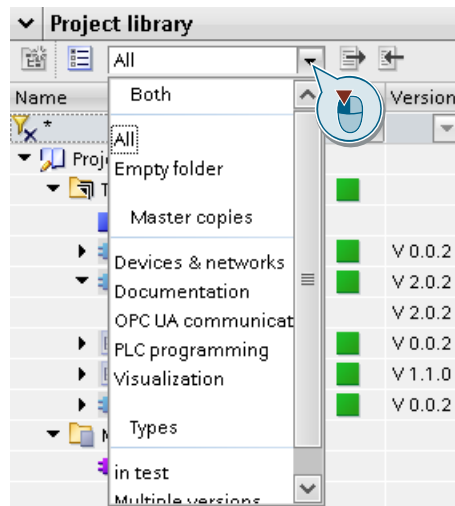
8.5.4 Filtering library elements in the project library

To maintain a clean overview of a large number of library elements, the "Libraries" task card offers various ways of filtering the display. The filters can also be combined.

Filter templates

With filter templates it is possible, for example, to show all types that are [in test].

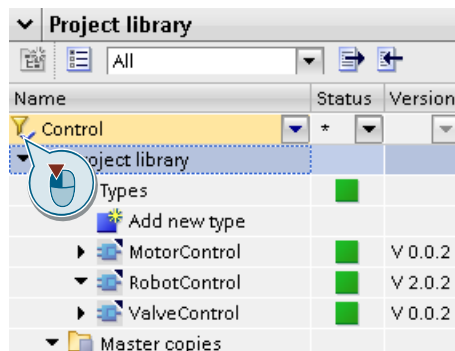
Figure 8-3: Filter templates



Text filter

You can enter a search term in the search field in the "Name" column to display library elements, for example, that contain the search term in their name. You can also use the text search for versions. Click on the filter icon to disable or re-enable the filter.

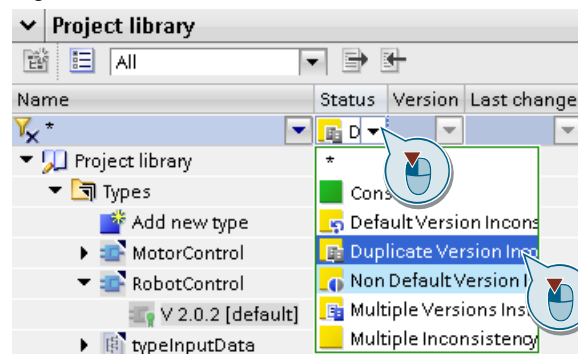
Figure 8-4: Text filter



Status filter

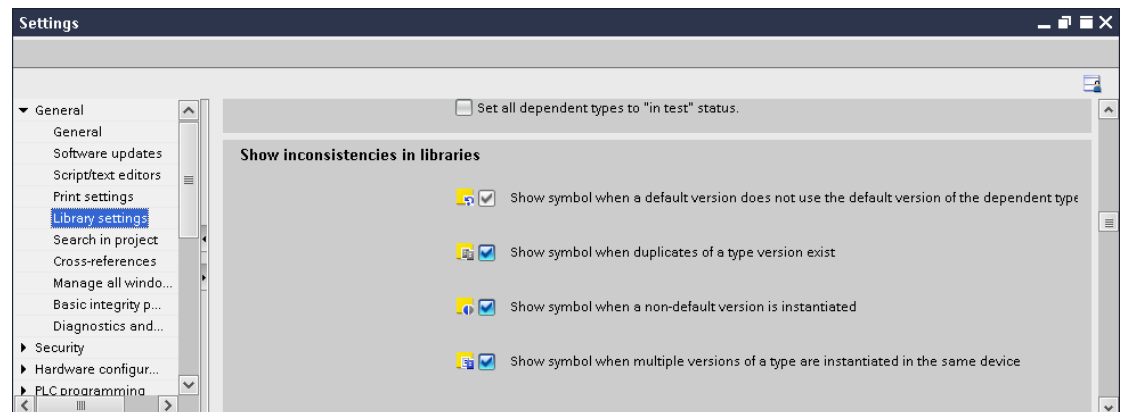
Filters rapidly give you an overview of the library elements which are inconsistent or which have two identical versions, for example.

Figure 8-5 Status filter



In the library settings, it is possible to define which status will be defined.

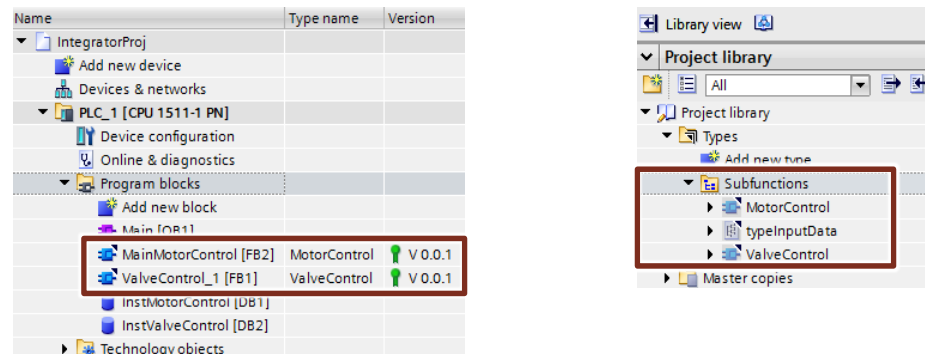
Figure 8-6: Library settings



8.5.5 Harmonize project and libraries

If you have renamed the types in the program or if they were automatically renamed by the TIA Portal update mechanisms, you have the ability to compare the names and paths of the project library with the programs in the project.

Figure 8-7: Type instance names do not match the type names or folder structure

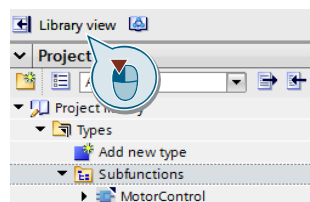


Solution

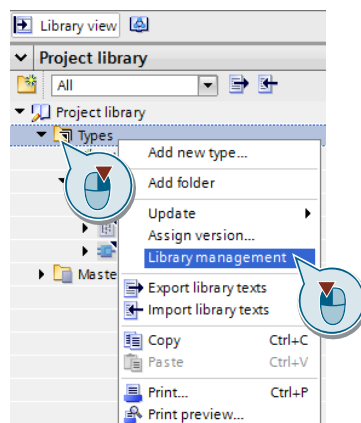
Using the central harmonization function in the Library management, you can harmonize all type instances in all devices as follows.

- Type names will be compared to the type instance names.
- Project library paths will be compared to the program.

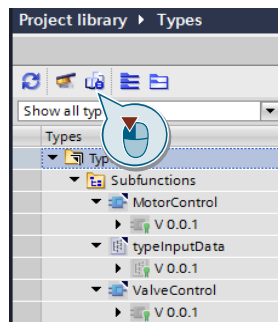
1. Open the "Library view".



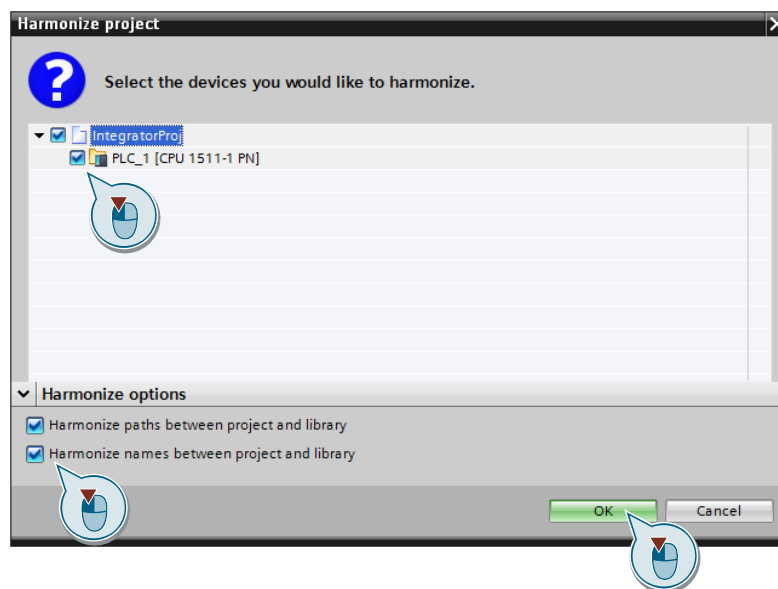
2. Right-click on the "Types" folder in the project library and select "Library Management".



3. Click "Harmonize project".



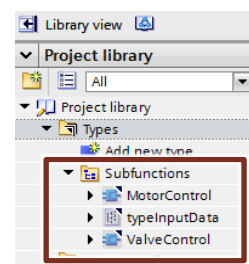
4. Select the PLC where you want to harmonize the type instances. Individually select whether the paths and/or names should be modified. Confirm the dialog with "OK".



Result

- Type instance names match the type names.
- Type instances are in the same folder structure as the types in the project library.

Name	Type name	Version
IntegratorProj		
Add new device		
Devices & networks		
PLC_1 [CPU 1511-1 PN]		
Device configuration		
Online & diagnostics		
Program blocks		
Add new block		
Main [OB1]		
InstMotorControl [DB1]		
InstValveControl [DB1]		
Subfunctions		
MotorControl [FB2]	MotorControl	V 0.0.1
ValveControl [FB1]	ValveControl	V 0.0.1



8.6 Version numbers

The version of a library element always consists of three numbers. The following table shows a suggestion of how you can assign version numbers.

Table 8-3: Version definition

Vx.y.z	Major version number	Minor version number	Revision number
	Non-compatible change	Compatible change	Error correction
	- Reduction of interfaces - Change to the existing interfaces - Incompatible extension of functionality	- Extension of the interfaces - Compatible extension of functionality	Bug fix

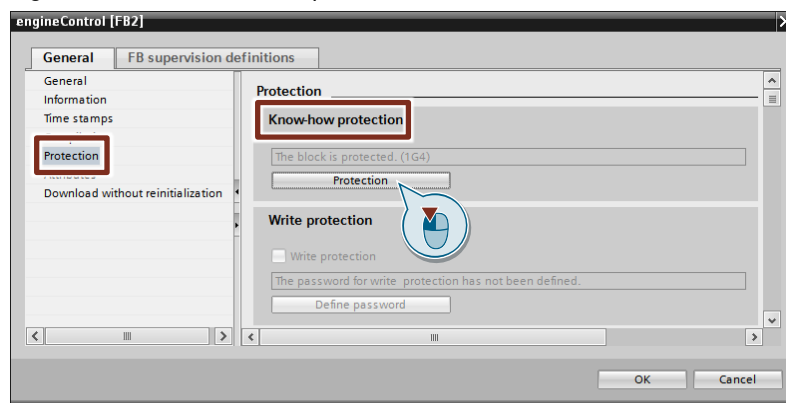
8.7 Know-how protection

Know-how protection allows you to protect blocks with a password against unauthorized access.

The blocks have the following properties in global libraries:

- Know-how-protected blocks can be typified.
- Know-how-protected blocks can only be re-compiled with the password.

Figure 8-8: Block know-how protection



Note You can find additional information in the TIA Portal Online Help under "Programming PLCs > Protect blocks > Block protections".

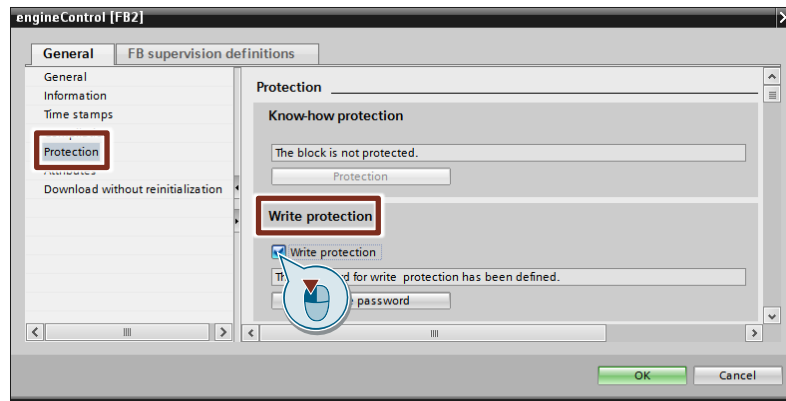
Note Know-how-protected blocks with SIMATIC S7-1200
Know-how-protected blocks can only be downloaded to the PLC without re-compiling (which requires a password) within the major version of the firmware. This only applies for S7-1200 PLCs.

8.8 Write protection

You can set up password-based write protection for blocks to prevent unintended modification. Blocks with write protection can only be opened in read-only mode, but you can continue editing and compiling the block properties.

Be aware that write protection is not know-how protection, as you can remove the write protection at any time. When a block is write-protected, you cannot set up know-how protection on top of this.

Figure 8-9: Block write protection



8.9 User-defined documentation (User help)

You can create your own user-defined documentation to explain to others how your project functions or how to use individual library types. The user help supports the pdf, chm, docx, xlsx, pptx, html xps, rtf and txt file formats.

8.9.1 Creating user-defined documentation

User-defined documentation supports multiple file types.

Recommendation: Create the user-defined documentation as a PDF file, for example, so that the user help can be opened independent of platform on many different computers.

The PDF files for user-defined documentation of individual blocks in the library will be created outside of TIA Portal. You can create the user help in all available interface languages.

In order to retrieve user help for the right block, the file name must match the name of the block in the TIA Portal exactly, e.g. "LGF_Astro.pdf" for the block "LGF_Astro". In addition, the PDF files must be saved in the correct directory.

So that user-defined documentation for library blocks can be delivered along with the library, save the PDF files in the following global library directory.

```
<Folder of the global library>\UserFiles\UserDocumentation\<Folder for the corresponding language>\<Object category>
```

The following table shows the folders for each language for the default interface languages:

Table 8-4

Language	Sub-folder for the respective language
German	\de-DE
English	\en-US
Spanish	\es-ES
French	\fr-FR
Italian	\it-IT
Chinese	\zh-CN

The following table shows **only** the English names of the most important object categories for libraries in TIA Portal:

Table 8-5

Object category	English name of the sub-folder
Types in the library	Library types
Master copies in the library	Master Copies
Libraries in the "Libraries" task card or in the library view	Libraries

Example: Directory for a type in a library

The German and English PDF files for the types in the "LGF" library are saved in the following directories of the global library.

- German: "LGF\UserFiles\UserDocumentation\de-DE\Library Types"
- English: "LGF\UserFiles\UserDocumentation\en-US\Library Types"

Example: Directory for master copies in a library

The German and English PDF files for the master copies in the "LGF" library are saved in the following directories of the global library.

- German: "LGF\UserFiles\UserDocumentation\de-DE\Master Copies"
- English: "LGF\UserFiles\UserDocumentation\en-US\ Master Copies "

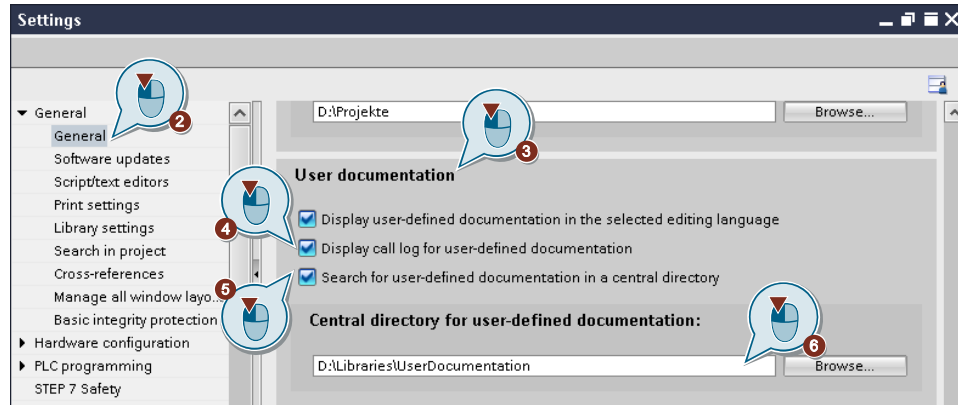
Alternative: Central directory for user-defined documentation

Alternatively, you can store the user-defined documentation in a central directory for all projects. The same rules that apply to the sub-folders of the global library also apply to the structure of sub-folders in the central directory.

To define a central storage location for user help, proceed as follows:

1. In the "Extras" menu, select the "Settings" command.
2. Open the area "General > General".
3. Navigate to the "User documentation" section.

4. Enable the check box "Display call log for user-defined documentation" to display a log of the call-up of the user-defined documentation in the Inspector window.
5. Enable the "Search for user-defined documentation in a central directory" check box to store user-defined documentation in a central directory for projects.
6. In the "Central directory for user-defined documentation" field, specify the path where you want to store cross-project documentation.



Note

To make it easier to connect to user-defined documentation, an access log (see Point 4) can be shown for the user-defined documentation. The messages in the log show which directories are being searched for documentation and whether the user-defined documentation has been successfully accessed. The expected file name is also shown. Here you can see how you must name your user-defined documentation and in which directories you have to save it. The access log matches the same order in which the user-defined documentation is searched.

8.9.2 Accessing user-defined documentation

You can access the user-defined documentation for a selected block in the "Library" task card or in the library view with the key combination <Shift+F1>. The respective PDF file is always opened with the default program in Microsoft Windows.

So that the user-defined documentation for the blocks can also be accessed in the project tree, you must copy the directories with PDF files into the project directory "UserFiles" or into the central directory for user-defined documentation (see [Alternative: Central directory for user-defined documentation](#)).

8.10 Enabling simulation support for library blocks

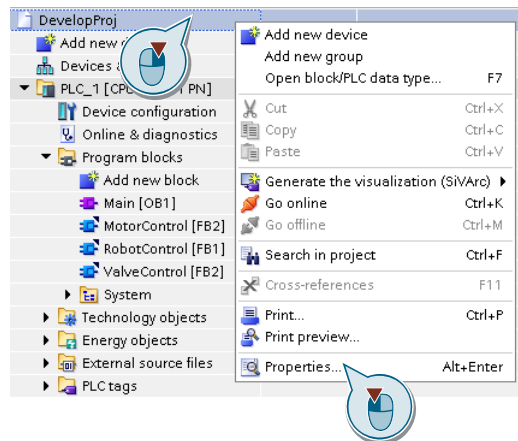
Requirements

All library blocks are instantiated.

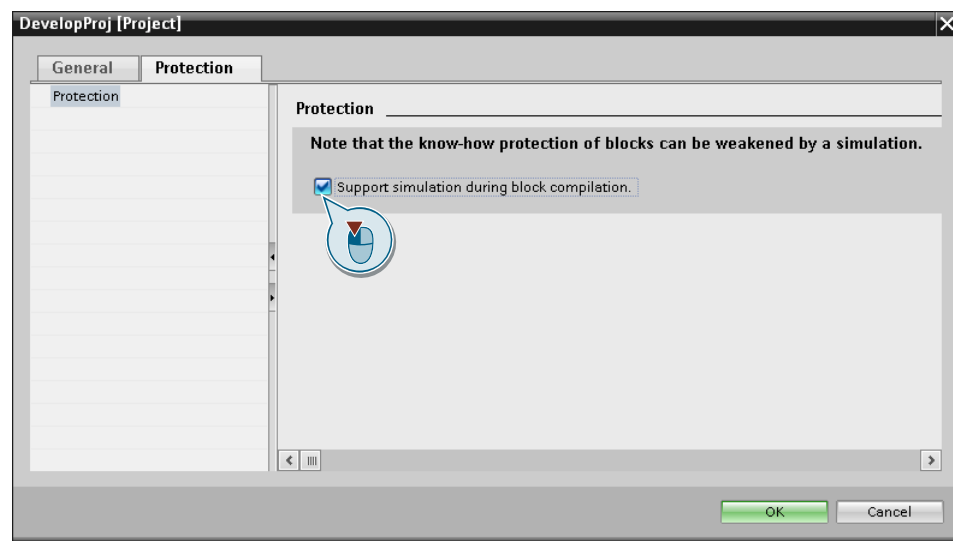
Procedure

Proceed as follows to enable simulation support for multiple library blocks:

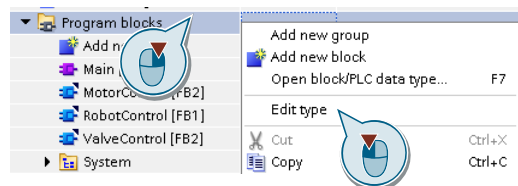
1. Open the project properties.



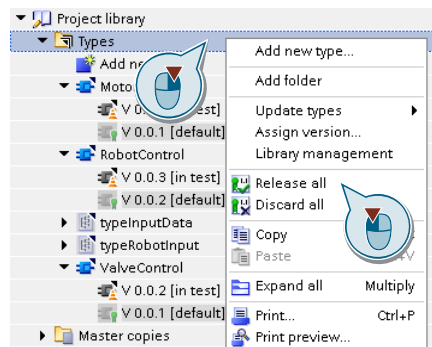
2. In the "Protection" tab, enable the option "Support simulation during block compilation".



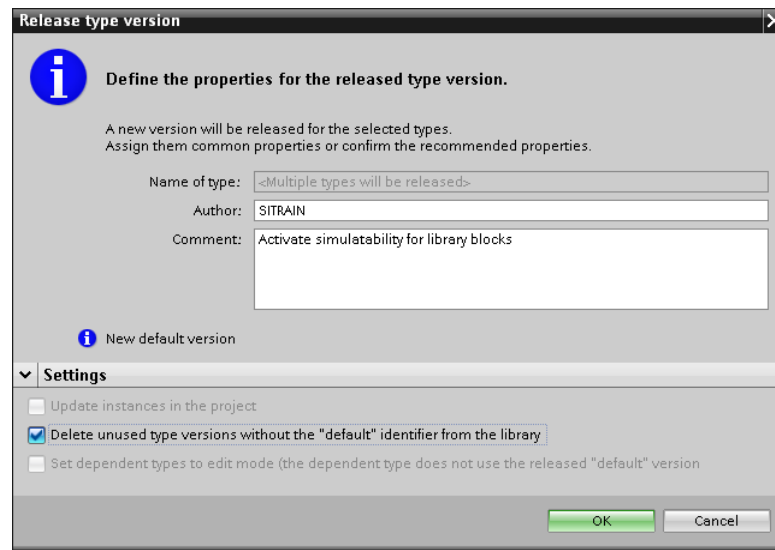
3. Right-click on "Program blocks" and select "Edit type".



4. Confirm the next dialog.
5. Right-click in the project library on the "Types" folder and press "Release all".



6. Update the metadata, enable the option ("Delete unused type versions without the "default" identifier from the library" and confirm the dialog with "OK".



9 Appendix

9.1 Service and support

Industry Online Support

Do you have any questions or need assistance?

Siemens Industry Online Support offers round the clock access to our entire service and support know-how and portfolio.

The Industry Online Support is the central address for information about our products, solutions and services.

Product information, manuals, downloads, FAQs, application examples and videos – all information is accessible with just a few mouse clicks:

support.industry.siemens.com

Technical Support

The Technical Support of Siemens Industry provides you fast and competent support regarding all technical queries with numerous tailor-made offers – ranging from basic support to individual support contracts.

Please send queries to Technical Support via Web form:

support.industry.siemens.com/cs/my/src

SITRAIN – Digital Industry Academy

We support you with our globally available training courses for industry with practical experience, innovative learning methods and a concept that's tailored to the customer's specific needs.

For more information on our offered trainings and courses, as well as their locations and dates, refer to our web page:

siemens.com/sitrain

Service offer

Our range of services includes the following:

- Plant data services
- Spare parts services
- Repair services
- On-site and maintenance services
- Retrofitting and modernization services
- Service programs and contracts

You can find detailed information on our range of services in the service catalog web page:

support.industry.siemens.com/cs/sc

Industry Online Support app

You will receive optimum support wherever you are with the "Siemens Industry Online Support" app. The app is available for iOS and Android:

support.industry.siemens.com/cs/ww/en/sc/2067

9.2 Industry Mall



The Siemens Industry Mall is the platform on which the entire Siemens Industry product portfolio is accessible. From the selection of products to the order and the delivery tracking, the Industry Mall enables the complete purchasing processing – directly and independently of time and location:

mall.industry.siemens.com

9.3 Links and literature

Table 9-1

No.	Subject
\1\	Siemens Industry Online Support https://support.industry.siemens.com
\2\	Link to the article page of the application example https://support.industry.siemens.com/cs/ww/en/view/109747503
\3\	FAQ: When starting TIA Portal V13 and higher, how do you get a global library to open automatically and use it as corporate library, for example? https://support.industry.siemens.com/cs/ww/en/view/100451450
\4\	Topic page: Libraries in TIA Portal https://support.industry.siemens.com/cs/ww/en/view/109738702
\5\	Programming style guide for S7-1200 and S7-1500 https://support.industry.siemens.com/cs/ww/en/view/81318674
\6\	SIMATIC S7-PLCSIM Advanced: Co-simulation via API https://support.industry.siemens.com/cs/ww/en/view/109739660
\7\	Library with general functions (LGF) for STEP 7 (TIA Portal) and S7-1200 / S7-1500 https://support.industry.siemens.com/cs/ww/en/view/109479728
\8\	FAQ: How do you open, edit and upgrade global libraries in the TIA Portal? https://support.industry.siemens.com/cs/ww/en/view/37364723
\9\	FAQ: How can you open a global library with write permissions in STEP 7 (TIA Portal)? https://support.industry.siemens.com/cs/ww/en/view/37364723
\10\	FAQ: How do you update corporate libraries in STEP 7 (TIA Portal)? https://support.industry.siemens.com/cs/ww/en/view/109739199

9.4 Change documentation

Table 9-2

Version	Date	Change
V1.0	09/2017	First version
V1.1	10/2019	New chapters 3.3.2 Creating write-protected libraries 3.3.3 Discontinuation of types 5.1.5 Replacing discontinued types 8.9 User-defined documentation (User help)
V1.2	12/2019	Added to chapter 5.1.4 Updating all types in the "plant project"
V1.3	09/2021	Update for TIA Portal V17